

[NAME OF YOUR INSTITUTE]

[Affiliated University / Board]

[DEPARTMENT NAME]

Project Report on

JAY RANCHHOD GAS SERVICE

Daily Operations & Inventory Management System

A Full-Stack Web Application for LPG Distributorship Automation

Submitted By

Name of Student : [STUDENT NAME 1]

Enrollment No : [ENROLLMENT NUMBER 1]

Name of Student : [STUDENT NAME 2]

Enrollment No : [ENROLLMENT NUMBER 2]

Name of Student : [STUDENT NAME 3]

Enrollment No : [ENROLLMENT NUMBER 3]

Guided By

[NAME OF INTERNAL GUIDE]

[SUBJECT NAME] ([SUBJECT CODE])

A.Y. [ACADEMIC YEAR] [TERM]

CERTIFICATE

This is to certify that the Project Report entitled “Jay Ranchhod Gas Service — Daily Operations & Inventory Management System” has been carried out by [STUDENT NAME 1] ([ENROLLMENT NUMBER 1]), [STUDENT NAME 2] ([ENROLLMENT NUMBER 2]) and [STUDENT NAME 3] ([ENROLLMENT NUMBER 3]) under my guidance in partial fulfilment for the Degree of [DEGREE NAME] in [BRANCH / SPECIALISATION], [SEMESTER] Semester of [NAME OF YOUR INSTITUTE], [AFFILIATED UNIVERSITY], during the academic year [ACADEMIC YEAR].

The work embodied in this report is original and has been carried out by the candidates themselves. To the best of my knowledge, the content of this report has not been submitted elsewhere for the award of any other degree or diploma.

[INTERNAL GUIDE NAME]

Internal Guide

[HEAD OF DEPARTMENT NAME]

Head of Department

DECLARATION

We hereby declare that the project report entitled “Jay Ranchhod Gas Service — Daily Operations & Inventory Management System” submitted to [NAME OF YOUR INSTITUTE] in partial fulfilment of the requirements for the award of the degree of [DEGREE NAME] is a record of original work carried out by us under the supervision of [NAME OF INTERNAL GUIDE].

We further declare that the analysis, design, source code, database schema and results presented in this report are the outcome of our own effort. Any material taken from published or online sources has been duly acknowledged and cited in the references section. This work has not previously formed the basis for the award of any degree, diploma, associateship or fellowship of any other university or institution.

The software described in this report has been developed for and deployed at a working LPG distributorship, and the functional requirements documented herein were gathered directly from the day-to-day operations of that business.

Signature of Students:

1. [STUDENT NAME 1]

2. [STUDENT NAME 2]

3. [STUDENT NAME 3]

Place: [CITY]

Date: [DD / MM / YYYY]

ACKNOWLEDGEMENT

We extend our heartfelt gratitude to all those who have contributed to the successful completion of this project on “Jay Ranchhod Gas Service — Daily Operations & Inventory Management System.”

First and foremost, we express our sincere appreciation to our project guide, [NAME OF INTERNAL GUIDE], whose guidance, patience and technical insight have been invaluable throughout the development of this system. Their suggestions on data modelling, transactional integrity and user-interface design substantially shaped the final architecture of the application, and their insistence on rigorous testing saved us from several costly design mistakes.

We would also like to thank the Head of the Department and the faculty members of our department for providing the laboratory infrastructure, database servers and academic support required during the development period. We are especially grateful for the freedom we were given to select a live, real-world problem rather than a purely academic exercise.

Our sincere thanks go to the proprietor and staff of Jay Ranchhod Gas Service, the Bharat Gas LPG distributorship whose daily operations form the basis of this project. They patiently walked us through their existing register-based bookkeeping, explained the commercial logic behind SBC and DBC schemes, credit sales, delivery-boy accounting and empty-cylinder reconciliation, and permitted us to observe a full working day at the counter and the godown. Without their cooperation, the domain model presented in Chapter 4 would have been mere guesswork.

We are grateful to our peers and classmates for their cooperation, constructive feedback and collaborative spirit, which played an important role in refining our ideas and improving the usability of the interface. Several of them acted as volunteer testers during the black-box testing phase.

Furthermore, we acknowledge the contributions of the open-source community. The React, Vite, Node.js, Express, MySQL, mysql2 and SweetAlert2 projects, together with the extensive documentation maintained by MDN Web Docs, formed the technical foundation upon which this application was built.

Finally, we would like to thank our families for their constant encouragement and support, which motivated us to complete this project successfully.

ABSTRACT

Jay Ranchhod Gas Service is a Bharat Gas LPG distributorship whose entire daily accounting — cylinder stock, cash reconciliation, credit sales, delivery-boy settlements, vehicle expenses and staff salaries — was historically maintained in handwritten registers. This project delivers a complete, production-deployed, full-stack web application that replaces those registers with a single transactional system of record.

The system is built on a conventional but rigorously applied three-tier architecture. The presentation tier is a React 19 single-page application bundled with Vite 8, structured around a central state container that hydrates itself from one consolidated master-load endpoint. The application tier is a Node.js service built on Express 5, exposing twenty-four RESTful endpoints protected by stateless JSON Web Token authentication with a one-hour session lifetime. The data tier is a MySQL 8 relational database of nineteen normalised tables, accessed through a pooled, prepared-statement interface with SSL transport and a self-executing schema-migration routine that keeps deployed instances in step with the application version.

The functional core of the system is a single atomic “daily entry” transaction. In one screen the counter operator records opening cash, vehicle arrivals from the bottling plant, per-product cash and online sales, subsidised (SBC) and domestic (DBC) scheme cylinders, the delivery-boy cash and online split, accessory sales, sundry expenses, cheque and online receipts, credit sales, credit recoveries, vehicle running expenses, salary and advance payments, sundry cash credits and physical godown stock. When the entry is saved, the server executes twelve sequential insert, update and delete operations inside a single SQL transaction, guaranteeing that a day is either recorded completely or not at all.

Two derived quantities are computed automatically and form the analytical heart of the application. The first is the closing full-cylinder stock, obtained from the opening stock plus plant arrivals less every category of outward movement; this value becomes the following day’s opening stock, producing a continuous, self-carrying inventory chain. The second is the In-Out Stock Master empty-cylinder balance, computed as a running total across the entire history of entries, which reconciles empties received from customers against empties

despatched to the bottling plant. A parallel cash formula derives closing cash on hand from opening cash, sales, recoveries and sundry credits net of expenses, salaries and bank deposits.

Access is partitioned into two operational roles. The counter operator is restricted to the current working date and to five functional tabs; the administrator has eleven tabs, may edit or delete back-dated entries, and controls the effective-dated price and commission histories, the vehicle and employee masters and the system user list. Because prices and commission rates are stored as effective-dated history rather than as mutable fields, every historical entry can be re-priced correctly at the rate that applied on the day it was recorded.

The system has been deployed to a managed cloud platform and is in live daily use. Under operational testing the consolidated master-load endpoint returned a complete working dataset in under two seconds, the daily-entry transaction committed in well under one second, and the automatically derived closing stock and cash-on-hand figures matched the manually maintained registers exactly over the verification period. The result is a measurable reduction in end-of-day reconciliation effort, elimination of arithmetic error in cylinder and cash accounting, and immediate visibility of outstanding customer credit that was previously buried in a paper ledger.

TABLE OF CONTENTS

Chapter	Page No.
Certificate.....	ii
Declaration.....	iii
Acknowledgement.....	iv
Abstract.....	v
Table of Contents.....	vi
List of Figures.....	vii
List of Tables.....	viii
1 Introduction.....	1
1.1 Problem Statement.....	1
1.2 Project Summary and Introduction.....	3
1.3 Aim and Objectives of the Project.....	5
1.4 Scope of the Project.....	7
2 System Analysis.....	9
2.1 Motivation.....	9
2.2 Brief Literature Survey.....	11
2.3 Study of the Existing Manual System.....	14
2.4 Limitations of the Existing System.....	15
2.5 The Proposed System.....	16
2.6 Feasibility Study.....	17
2.7 Requirement Specification.....	19
3 Design: Analysis, Design Methodology.....	22
3.1 Hardware and Software Requirements.....	22
3.2 Program / Module Specification.....	24
3.3 Timeline Chart.....	29
3.4 UML and Structured Design Diagrams.....	30

4	Implementation.....	37
4.1	System Flow.....	37
4.2	Database Design.....	39
4.3	The Business Calculation Engine.....	45
4.4	Module Specification (Screenshots of Working Modules).....	48
4.5	Testing.....	54
4.6	Results.....	57
	Conclusion and Future Work.....	59
	References.....	62

LIST OF FIGURES

Fig 1: Project Development Timeline (Gantt Chart).....	29
Fig 2: Flowchart Diagram — Authentication and Daily Entry Lifecycle.....	31
Fig 3: Use Case Diagram.....	32
Fig 4: Level 0 Data Flow Diagram (Context Diagram).....	33
Fig 5: Level 1 Data Flow Diagram.....	34
Fig 6: Entity Relationship Diagram.....	35
Fig 7: Sequence Diagram — Login, Master Load and Entry Persistence.....	36
Fig 8: System Flow / Three-Tier Architecture Diagram.....	38
Fig 9: Login Screen.....	48
Fig 10: Daily Entry — Header, Vehicle Arrival and Product Stock.....	49
Fig 11: Daily Entry — Delivery Boy Split and Accessory Sales.....	49
Fig 12: Daily Entry — Expenses, Cheque / Online and Credit Sales.....	50
Fig 13: Daily Entry — In-Out Stock Master and Cash Summary.....	50
Fig 14: History Tab — Consolidated Day Register.....	51
Fig 15: Pending Credits and Recovery Recording.....	51
Fig 16: Summary Tab — Period Sales Analysis.....	52
Fig 17: Admin Dashboard — Key Performance Indicators.....	52
Fig 18: Admin — Price History and Commission Management.....	53
Fig 19: Admin — Vehicle Master and Employee Master.....	53
Fig 20: Admin — Day Report, Salary Report and Credit Ledger.....	54

LIST OF TABLES

Table 1: Comparison of the Manual Register System and the Proposed System.....	16
Table 2: Functional Requirements.....	19
Table 3: Non-Functional Requirements.....	21
Table 4: Client-Side Hardware Requirements.....	22
Table 5: Server-Side and Development Environment Requirements.....	23
Table 6: Software Stack and Version Matrix.....	23
Table 7: Frontend Module Specification.....	25
Table 8: Backend REST API Endpoint Specification.....	27
Table 9: Project Phase Breakdown.....	29
Table 10: Database Table Inventory by Functional Group.....	39
Table 11: Data Dictionary — Master and Reference Tables.....	40
Table 12: Data Dictionary — Daily Transaction Tables.....	42
Table 13: Data Dictionary — Stock, Credit and Payroll Tables.....	44
Table 14: Product Catalogue and Scheme Codes.....	45
Table 15: Unit and Integration Test Cases.....	55
Table 16: Performance Measurements.....	57
Table 17: Operational Impact Assessment.....	58

Chapter 1: Introduction

1.1 Problem Statement

A Liquefied Petroleum Gas (LPG) distributorship occupies an unusual position in Indian retail. It is simultaneously a cash business, a credit business, a physical inventory business and a reverse-logistics business, and it is obliged to reconcile all four every single day before the counter closes. Jay Ranchhod Gas Service, a Bharat Gas distributorship, handles three distinct cylinder variants — the 14.2 kg domestic cylinder, the 19 kg commercial cylinder and the 5 kg FLT portable cylinder — across a customer base that includes walk-in domestic households, commercial establishments on running credit accounts, and a fleet of delivery personnel who take stock out in the morning and return with cash, digital-payment confirmations and empty cylinders in the evening.

Before the development of this system, every one of those movements was recorded by hand. A bound register held the day's opening cylinder count; a second register held the cash book; a third held the credit khata for customers who took cylinders on account; loose sheets tracked delivery-boy settlements; and the empty-cylinder count returned to the bottling plant was reconciled from delivery challans kept in a file. The problems that arise from such an arrangement are not merely inconvenient — they are structural, and they compound.

1.1.1 Arithmetic Error in a Chain-Dependent Ledger

The most damaging characteristic of cylinder accounting is that it is chain-dependent. The closing stock of one day is definitionally the opening stock of the next. When a clerk makes a single subtraction error on a Tuesday, that error does not remain confined to Tuesday; it silently propagates through Wednesday, Thursday and every subsequent day until somebody performs a full physical count and forces a correction. Because a physical count of a godown holding several hundred cylinders is time-consuming, such counts were performed infrequently, and by the time a discrepancy surfaced it was impossible to determine on which day it had originated or how large the true shortfall was.

The same chain-dependency applies to cash. Closing cash on hand is opening cash plus receipts less disbursements. A misposted expense on any given day produces a cash figure that is wrong on that day and on every day thereafter, and the only remedy is a physical cash count that reveals the aggregate discrepancy without localising it.

1.1.2 Fragmentation of the Empty-Cylinder Cycle

An LPG distributorship does not sell cylinders; it sells the gas within them and exchanges the steel container. Every filled cylinder that leaves the premises should, in principle, be matched by an empty cylinder returning — either immediately from a walk-in customer, or later from a credit customer, or over time from a delivery route. Those accumulated empties are then despatched back to the bottling plant when the next filled-cylinder vehicle arrives.

In the manual system this cycle was recorded in at least three different places: the counter register noted cash exchanges, the credit khata noted cylinders held against account, and the plant challan noted the despatch quantity. No single document reconciled the three. The distributorship therefore had no reliable running figure for how many empty cylinders it was actually holding, and disputes with the bottling plant over despatch quantities could not be settled from the agency's own records.

1.1.3 Opacity of Outstanding Credit

Commercial customers frequently take cylinders on credit, paying in irregular instalments and returning empties on a schedule that does not align with their payments. The paper khata recorded each sale and each payment, but computing the true outstanding balance for a given customer required manually summing every credit entry and subtracting every recorded payment — an operation nobody performed routinely. Ageing analysis, the identification of dormant defaulters, and the tracking of empty cylinders still held by credit customers were all effectively impossible.

1.1.4 Absence of Effective-Dated Pricing

LPG retail prices in India are revised periodically by the oil marketing companies, and the distributor's commission per cylinder is revised on an independent schedule. A register records only the rupee amount collected; it does not record the rate that produced that amount. Consequently, any attempt to recompute historical revenue, verify commission entitlement for a past month, or analyse the margin impact of a price revision required cross-referencing the registers against circulars — a task that was never undertaken in practice.

1.1.5 Loss of Managerial Visibility

Finally, because all data resided on paper at the counter, the proprietor had no ability to answer routine managerial questions without physically visiting the premises and reading through registers. Questions as basic as “how many 14 kg cylinders did we sell this month compared with last month”, “what is our total outstanding credit today”, or “how much salary advance

has this employee already drawn this month” required an afternoon of manual tabulation. The information existed, but it was not accessible, and information that is not accessible cannot inform a decision.

1.2 Project Summary and Introduction

This project is a complete, production-deployed, full-stack web application that replaces the entire manual bookkeeping apparatus of an LPG distributorship with a single transactional system of record. It is not a prototype and not a simulation: the system runs on managed cloud infrastructure, is accessed daily by the counter staff and the proprietor of a working Bharat Gas agency, and is the authoritative record of that agency’s stock and cash position.

The application is constructed on a conventional three-tier architecture, chosen deliberately for its operational robustness rather than for novelty. The presentation tier is a single-page application written in React 19 and bundled with Vite 8. It is organised around one global state container which, immediately after authentication, hydrates itself from a single consolidated endpoint and thereafter serves every screen from client memory. This design was adopted because the counter operates over an intermittent mobile-broadband connection where a chatty, request-per-screen interface would be unusable.

The application tier is a Node.js service built on Express 5. It exposes twenty-four RESTful endpoints covering authentication, consolidated data loading, daily-entry persistence, credit-recovery recording, effective-dated price and commission synchronisation, and full create-read-update-delete management of the vehicle master, employee master, godown stock and system user list. Authentication is stateless and token-based: a successful login returns a signed JSON Web Token with a one-hour expiry, which the client attaches as a bearer credential to every subsequent request and which a single middleware function validates before any protected handler executes.

The data tier is a MySQL 8 relational database comprising nineteen normalised tables, accessed through a pooled, prepared-statement interface with SSL transport. A schema-migration routine executes automatically at server start-up, inspecting the live schema and applying any missing columns or tables idempotently, so that a deployment of a newer application version brings its own database changes with it and cannot be run against a stale schema.

The functional centre of the application is a single screen — the Daily Entry — which corresponds to one calendar day of trading and to one atomic database transaction. On this screen the operator records, in one pass: opening cash carried forward; whether a cylinder vehicle arrived from the bottling plant and if so the filled cylinders received and empty

cylinders despatched per product; per-product cash sales, online sales, and SBC and DBC scheme cylinders with their independent rate schedules; the cash and online split for each delivery boy; accessory sales of gas pipes and stoves; itemised sundry expenses; cheque and online receipts; credit sales identifying customer, product, filled and empty cylinder quantities and amount; recoveries against previously issued credit including any empty cylinders returned with the payment; vehicle running expenses categorised by type and attributed to a specific vehicle in the fleet; salary and advance payments attributed to a specific employee and to a specific salary month; sundry cash credits; and the physically counted godown stock of filled and empty cylinders.

When the operator saves, the server opens a SQL transaction and executes twelve sequential persistence steps across twelve tables — upserting the day header, replacing the product stock rows, resolving delivery-boy names to employee identifiers, rewriting the expense, cheque, vehicle-expense, salary, accessory and sundry-credit collections, reconciling the credit ledger against the submitted list while protecting any ledger row that already carries a payment, and upserting the godown and arrival records. Either all twelve steps commit or the transaction is rolled back in its entirety. There is no state in which a day is half-recorded.

Two derived quantities are computed automatically from this data and constitute the analytical value of the system. The closing full-cylinder stock is calculated as the opening stock plus plant arrivals less cash sales, online sales, SBC cylinders, DBC cylinders and cylinders issued on credit; this figure is written to the database and becomes the following day's opening stock, producing a continuous self-carrying inventory chain that no longer depends on a clerk performing a subtraction correctly. The In-Out Stock Master empty-cylinder balance is computed as a running total across the complete history of entries, adding every empty received from a sale, a credit transaction or a recovery, and subtracting every empty despatched to the bottling plant, thereby providing for the first time a single authoritative empty-cylinder position.

Access to the system is partitioned into two roles. The counter operator sees five tabs and is constrained to the current working date, preventing accidental or unauthorised alteration of closed days. The administrator sees eleven tabs, may open and edit any historical date, may delete a day's operational entry while preserving the permanent financial records attached to it, and controls the effective-dated price and commission tables, the vehicle and employee masters, and the system user list.

1.3 Aim and Objectives of the Project

The overarching aim of this project is to replace the manual, register-based daily accounting of an LPG distributorship with a single, transactionally safe, role-partitioned web application that computes inventory and cash positions automatically and makes the resulting information immediately available to management. Within that aim, the project pursues a set of specific and independently verifiable objectives.

Objective 1 — Establish a Single Atomic Unit of Record

The first objective is to collapse the several parallel registers into one logical daily entry, and to persist that entry as a single database transaction spanning all twelve affected tables. The success criterion is that no partial day can exist in the database: a save either records opening cash, product movement, deliveries, expenses, credit activity, payroll, accessories and godown stock together, or records none of them.

Objective 2 — Eliminate Manual Stock Arithmetic

The second objective is to remove human subtraction from the inventory chain entirely. The closing full-cylinder stock must be derived by the application from opening stock, plant arrivals and every outward movement category, and must be automatically carried forward as the next day's opening stock. The success criterion is that an operator never types a closing stock figure and that the carried-forward opening stock of any day equals the computed closing stock of the preceding day for every product.

Objective 3 — Reconcile the Empty-Cylinder Cycle

The third objective is to produce, for the first time, an authoritative running balance of empty cylinders held. This requires capturing empties from three previously disconnected sources — the exchange accompanying an ordinary sale, the empties recorded against a credit sale, and the empties returned alongside a credit recovery — and netting them against the empties despatched to the bottling plant on each vehicle arrival. The success criterion is that the computed empty balance agrees with a physical godown count.

Objective 4 — Make Outstanding Credit Continuously Visible

The fourth objective is to convert the paper khata into a queryable ledger in which every credit sale carries its own recovery history, the outstanding balance is derived rather than stored, a ledger row clears itself automatically once cumulative recoveries meet or exceed the original amount, and any empty cylinders still held by the customer are surfaced alongside the monetary

balance. The success criterion is that the total outstanding figure is available on the dashboard at all times without any manual tabulation.

Objective 5 — Preserve Historical Pricing Fidelity

The fifth objective is to store product rates and distributor commission rates as effective-dated histories rather than as mutable current values, so that any past transaction can be re-evaluated at the rate that actually applied on its date. The success criterion is that revising a price today does not alter the computed revenue of any previously recorded day.

Objective 6 — Enforce Role-Based Operational Boundaries

The sixth objective is to separate routine data capture from administrative control. Counter staff must be able to record the current day's business but must not be able to alter closed days or master configuration; administrators must be able to correct historical entries, manage masters and control system access. The success criterion is that the two roles present materially different interfaces and capability sets after authentication.

Objective 7 — Deliver Actionable Management Reporting

The seventh objective is to convert the captured data into decision-grade information: month-to-date sales with a comparison against the preceding month, cylinder throughput by product, commission earned, total outstanding credit, per-day cash reconciliation, per-employee salary and advance position against base salary, and a consolidated credit ledger. The success criterion is that each of these figures is available on demand without any manual computation.

Objective 8 — Achieve Real Operational Deployment

The eighth and final objective is that the system should not remain a laboratory artefact. It must be deployed to accessible infrastructure, be usable over an ordinary mobile-broadband connection on both desktop and mobile viewports, and be adopted as the agency's working record. The success criterion is sustained daily use by the intended operators.

1.4 Scope of the Project

It is as important to state what a system does not do as what it does. The scope of this project was deliberately bounded to the internal daily operations of a single distributorship, and the boundaries were agreed with the business before development began.

1.4.1 Within Scope

- Complete daily transactional capture for the three cylinder products and two accessory products handled by the agency, including cash, online, SBC and DBC sale channels.
- Automatic derivation and carry-forward of closing full-cylinder stock, and running reconciliation of the empty-cylinder balance across the full history of entries.
- Recording of physically counted godown stock as an independent verification series against the computed In-Out Stock Master figure.
- Vehicle arrival capture, recording filled cylinders received from and empty cylinders despatched to the bottling plant.
- A complete credit ledger with per-sale recovery history, automatic clearance, derived outstanding balances and empty-cylinder tracking against credit customers.
- Delivery-boy accountability, recording the cash and online split of cylinders delivered by each named delivery person.
- Effective-dated price history with independent rates for the standard, SBC and DBC channels, and effective-dated distributor commission history.
- Vehicle master with fleet register and categorised running-expense capture attributed to individual vehicles.
- Employee master with base salary, and a payroll module recording salary and advance payments attributed to a specific salary month, with automatic balance computation.
- Cash-on-hand derivation from opening cash, all receipt categories and all disbursement categories including bank deposit.
- Two-role access control with distinct interfaces, and administrative management of system users.
- Management reporting comprising a KPI dashboard, per-day reports, period summaries, salary reports and a consolidated credit ledger.

1.4.2 Outside Scope

- Statutory accounting output. The system is an operational record, not a book of accounts; it does not generate GST returns, produce a trial balance, or integrate with commercial accounting packages.
- Customer-facing functionality. There is no customer portal, booking application or delivery-tracking interface; the system is used exclusively by agency staff.
- Integration with the oil marketing company's own distributor systems. Prices, commissions and plant arrivals are entered by the administrator rather than received electronically.

- Automated payment processing. Online receipts are recorded as amounts; the system does not connect to a payment gateway or reconcile against bank statements.
- Multi-branch or multi-agency operation. The data model assumes a single distributorship with a single godown.
- Barcode, RFID or IoT-based cylinder-level tracking. Inventory is tracked in aggregate quantities per product, not per individual cylinder serial number.

These exclusions were not arbitrary. Each represents either a regulatory domain better served by dedicated software, an integration dependent on a third party outside the project's control, or a hardware investment the business did not wish to make. Confining the project to the operational core allowed the available development effort to be spent on making that core correct and reliable rather than on making a broader system superficially complete.

Chapter 2: System Analysis

2.1 Motivation

The motivation for this project did not originate in a classroom exercise. It originated in an afternoon spent behind the counter of a working LPG distributorship, watching a clerk reconcile a day's trading with a pencil, a bound register and a pocket calculator, and observing how much of the working day that reconciliation consumed.

Three observations from that visit shaped everything that followed.

2.1.1 The Reconciliation Burden Is Daily and Non-Deferrable

Unlike many small-business bookkeeping tasks, LPG daily accounting cannot be postponed. Cash must be counted and agreed before the counter closes, because the cash drawer is physically emptied. Cylinder stock must be agreed because the godown is locked. Delivery boys must be settled because they go home. The reconciliation is therefore a mandatory, recurring, end-of-day cost imposed on the business every single trading day, and any reduction in that cost compounds three hundred times a year.

Measured over several observation days, the manual end-of-day reconciliation consumed between forty and seventy minutes, of which the greater part was arithmetic rather than counting. That arithmetic is precisely the category of work at which software excels and at which tired human beings, late in a long day, do not.

2.1.2 The Data Already Exists in Structured Form

A second observation was that the agency was not short of data. The registers contained, in disciplined and consistent handwriting, every figure that a management-information system would want. What was absent was not information but structure: the data was captured in a medium that permitted recording but not querying, aggregation or cross-referencing.

This is an important distinction, because it determined the character of the project. The system was not required to persuade the business to record new things; it was required to accept the things they already recorded and make those things computable. That framing substantially reduced the change-management risk, because the daily entry screen could be modelled directly on the layout of the existing register, and the operators could be trained in an afternoon.

2.1.3 Small Businesses Are Underserved by Available Software

A third motivation was a survey of what was commercially available. Enterprise resource planning suites capable of handling inventory, credit and payroll exist in abundance, but they

are priced, provisioned and configured for organisations very much larger than a single-godown gas agency. Their implementation typically requires a consultant, their licensing is recurring and substantial, and their generic inventory modules have no concept of the specific commercial mechanics of LPG distribution — the empty-cylinder exchange cycle, the SBC and DBC subsidy schemes, distributor commission per cylinder, or delivery-boy settlement.

Conversely, the free tools available to a small business — spreadsheets, most obviously — solve the arithmetic problem but not the integrity problem. A spreadsheet has no transactions, no role separation, no referential integrity, no audit position and no protection against a formula being overwritten by a value. It replaces one class of error with another.

The gap between these two extremes — a domain-specific, transactionally correct, role-partitioned system sized for a single small business — is precisely where this project sits, and that gap is the principal motivation for building rather than buying.

2.1.4 The Maturity of the Web Platform

A final motivating factor was technological. A decade ago, delivering a responsive, offline-tolerant, multi-user business application to a shop counter would have required installing and maintaining desktop software on specific machines. The maturation of the modern web platform has removed that requirement. A React application bundled by a modern build tool loads in under two seconds over mobile broadband, runs identically on the counter desktop and on the proprietor's telephone, requires no installation, and updates for every user the moment a new build is deployed. Managed cloud hosting for both the application server and the database has likewise fallen to a price point that a small business can absorb. The technical preconditions for a project of this kind are, for the first time, comfortably met.

2.2 Brief Literature Survey

The design of this system was informed by a review of published work in four areas: the general evolution of small-business inventory management systems, the specific literature on LPG and cylinder distribution, the architectural literature on transactional web applications, and the security literature on authentication in stateless application programming interfaces.

2.2.1 Inventory Management for Small and Medium Enterprises

The literature on inventory management in small and medium enterprises consistently identifies the same failure mode: the perpetual inventory record and the physical stock diverge over time, and the divergence is discovered late. Classical inventory theory, from the economic order quantity models onward, largely assumes that the recorded stock figure is correct and

concerns itself with when and how much to reorder. In practice, for small businesses, the harder problem is keeping the record accurate at all.

The remedy proposed across this literature is the perpetual system with periodic physical verification: every movement is recorded as it occurs, a computed balance is maintained continuously, and a physical count is performed at intervals to detect and correct drift. Crucially, the computed balance and the physical count are maintained as separate series so that the discrepancy itself is visible and measurable. This principle is directly implemented in the present system, where the In-Out Stock Master provides the computed perpetual balance and the godown stock module records the independent physical count, with any shortage recorded explicitly rather than being silently absorbed.

2.2.2 Reverse Logistics and Returnable Container Management

The empty-cylinder cycle is an instance of the returnable transport item problem, a well-studied area of reverse logistics. The literature on returnable containers — gas cylinders, beverage crates, industrial pallets, chemical drums — identifies a characteristic accounting difficulty: the container is an asset of the supplier that resides in the possession of the customer, and the supplier's books must therefore track a population of assets that is physically distributed and only intermittently observable.

Published approaches fall into two families. Serial-level tracking assigns an identity to every individual container, typically through barcode or RFID, and maintains a per-container custody record. Aggregate balance tracking maintains only counts, reconciling issues against returns. Serial-level tracking is more precise and supports individual accountability, but imposes hardware cost and a scanning discipline at every touchpoint. Aggregate tracking is cheaper and imposes no operational burden but cannot attribute a loss to a specific customer.

The present system adopts aggregate balance tracking as its primary mechanism, consistent with the scale and cost constraints of the business, but supplements it with partial attribution: empty cylinders associated with a credit sale are recorded against the named credit customer, so that the subset of outstanding empties that is most at risk of loss is individually attributed even though the general population is not.

2.2.3 Studies Specific to LPG Distribution

Literature specific to LPG distribution in the Indian context emphasises the regulatory and commercial complexity that generic inventory software does not accommodate. Distributors operate under commission structures set by the oil marketing companies rather than on a conventional buy-low-sell-high margin; subsidy schemes create parallel price channels for

what is physically the same product; and the distributor is accountable to the marketing company for cylinder custody in a manner that has no analogue in ordinary retail.

Studies of digitisation in this sector report that adoption failures are more often organisational than technical. Systems that require the counter to change its working sequence, or that demand data the counter does not naturally have at the moment of entry, are abandoned. Systems that mirror the existing register and simply automate its arithmetic are adopted. This finding directly influenced the interface design of the present project, in which the daily entry screen deliberately follows the section order of the agency's existing physical register.

2.2.4 Architectural Patterns for Transactional Web Applications

The architectural literature on line-of-business web applications converges on the layered or three-tier model: a presentation layer concerned only with rendering and input, an application layer holding business rules and transaction boundaries, and a data layer providing durable, consistent storage. The persistent value of this model lies in its clear allocation of the transaction boundary to the application layer, where it can span multiple storage operations.

Within this literature, the treatment of composite business documents is directly relevant. A daily entry is a composite document: a header with many dependent collections. Two persistence strategies are described. Differential update computes the delta between the submitted document and the stored document and applies only the changes; it is efficient but complex, and mis-computed deltas are a common source of defects. Replace-in-transaction deletes the dependent collections and re-inserts them wholesale within a single transaction; it is simpler, is trivially idempotent, and its correctness is easy to reason about, at the cost of writing more rows than strictly necessary.

The present system adopts replace-in-transaction for the ordinary dependent collections — expenses, cheques, vehicle expenses, salary payments, accessories and sundry credits — because the row counts involved are small and the simplicity is worth far more than the saved writes. It deliberately departs from this strategy for the credit ledger, where a naive delete-and-reinsert would destroy payment history; there, a reconciliation strategy is used that upserts submitted rows and deletes only those removed rows which carry no dependent payments.

2.2.5 Stateless Authentication and Session Management

The security literature contrasts server-side session storage with stateless token-based authentication. Server-side sessions place session state in the server's memory or in a shared store, and are straightforward to revoke; stateless tokens carry signed claims in the credential

itself, requiring no server-side lookup and scaling horizontally without a shared session store, at the cost of being difficult to revoke before expiry.

The consensus recommendation for token-based schemes is a short expiry window, which bounds the damage from a leaked credential without requiring a revocation infrastructure. The present system follows this recommendation with a one-hour token lifetime, validated by a single middleware function on every protected route, and supplemented by client-side interception of authentication failures so that an expired session produces an immediate, explanatory prompt to re-authenticate rather than an opaque failure.

2.3 Study of the Existing Manual System

A structured study of the incumbent manual system was conducted before any design work, comprising direct observation of a full trading day, interviews with the counter operator and the proprietor, and examination of the physical registers covering the preceding several months.

The manual system was found to consist of five distinct record-keeping artefacts. The daily sales register recorded, per product, the opening cylinder count, cylinders sold, and the closing count, together with the day's cash total. The cash book recorded opening cash, receipts, expenses and closing cash. The credit khata recorded, per customer, the date and amount of each credit sale and each subsequent payment. Loose settlement sheets recorded the cylinders allocated to and cash returned by each delivery boy. Finally, a file of plant challans documented each vehicle arrival, the filled cylinders received and the empty cylinders despatched.

The working sequence observed was as follows. Through the trading day, the counter operator recorded each sale in the daily sales register and each cash movement in the cash book, with credit sales additionally noted in the khata. In the evening, delivery boys returned and were settled against their allocation sheets. The operator then totalled the sales register, totalled the cash book, physically counted the cash drawer, and compared the two. Any difference was investigated by re-reading the day's entries. Once cash agreed, the closing cylinder count was computed by subtraction and carried forward by hand to the next page as the following day's opening figure.

Notably, no step in this sequence produced a reconciled empty-cylinder position, and no step produced an outstanding-credit total. Those figures were computed only when a specific question forced them to be, which in practice was rarely.

2.4 Limitations of the Existing System

The study identified the following specific limitations, each of which became a design requirement for the replacement system.

- **Propagating arithmetic error.** Because closing stock and closing cash are carried forward by hand, a single subtraction error persists indefinitely and cannot be localised after the fact.
- **No reconciled empty-cylinder balance.** Empties were recorded in three unconnected places and netted nowhere, leaving the agency unable to state its own empty holding or to substantiate a despatch dispute with the bottling plant.
- **Outstanding credit invisible without manual tabulation.** Deriving a customer balance required summing every khata line for that customer; a portfolio-wide total was effectively never computed.
- **Empty cylinders held by credit customers untracked.** The khata recorded money owed but not steel owed, so cylinders held against long-standing credit accounts were invisible.
- **No historical rate record.** Registers recorded amounts, not rates, making historical revenue recomputation and commission verification impossible.
- **No delivery-boy performance record.** Settlement sheets were discarded after settlement, so per-person delivery volumes could not be analysed.
- **Payroll advances untracked against entitlement.** Advances were paid from the drawer and noted informally; the running position of an employee against their monthly salary was not maintained.
- **Vehicle running costs unattributed.** Fuel and repair expenses were recorded as undifferentiated cash expenses, preventing any per-vehicle cost analysis.
- **Single point of physical failure.** The registers existed in one copy at one location, vulnerable to fire, water, loss or simple misplacement, with no backup of any kind.
- **No remote visibility.** The proprietor could not obtain any operational figure without physically attending the premises.
- **No access control.** Any person with physical access to a register could read or alter any historical entry, and no record of such alteration would exist.

2.5 The Proposed System

The proposed system addresses each identified limitation directly. The table below sets the two systems side by side across the operational dimensions that matter to the business.

Table 1: Comparison of the Manual Register System and the Proposed System

Operational Dimension	Existing Manual System	Proposed Web-Based System
Closing cylinder stock	Computed by hand and carried forward; errors propagate silently	Derived by the application from opening stock, arrivals and all outward movements; automatically carried forward
Empty cylinder balance	Not reconciled; recorded in three unconnected places	Running In-Out Stock Master balance netting all receipts against all plant despatches
Physical stock verification	Infrequent full godown counts with no recorded variance	Daily godown count captured as an independent series with explicit shortage recording
Cash on hand	Manually totalled and compared against a physical count	Derived from opening cash, all receipt categories and all disbursement categories
Outstanding credit	Requires manual summation of the khata per customer	Derived continuously per ledger row; portfolio total shown on the dashboard
Empties held by credit customers	Not recorded	Recorded per credit sale and reduced by empties returned with each recovery
Price and commission history	Not retained; only amounts recorded	Effective-dated history tables; any past day priced at its own applicable rate
Delivery boy accountability	Loose sheets discarded after settlement	Persistent per-employee cash and online delivery split for every day
Payroll position	Informal notes of advances paid	Salary and advance payments attributed to employee and salary month with derived balance
Vehicle running cost	Undifferentiated cash expense	Categorised expense attributed to a specific vehicle in the fleet master
Data durability	Single paper copy at a single location	Managed cloud database with provider-level backup and redundancy
Remote access	None	Any browser on any device over HTTPS
Access control	None; physical possession is authorisation	Token-based authentication with two roles and distinct capability sets
Management reporting	Ad hoc manual tabulation taking hours	Dashboard, day reports, period summaries, salary reports and ledger available on demand

2.6 Feasibility Study

A four-dimensional feasibility assessment was carried out before development commenced, covering technical, economic, operational and schedule feasibility.

2.6.1 Technical Feasibility

The technical demands of the system are modest by contemporary standards. The data volumes involved are small: a single distributorship generates one entry header, three product-stock

rows, a handful of delivery, expense and payment rows and a small number of credit transactions per trading day, amounting to a few thousand rows per year. No component of the workload approaches the capacity limits of a modest managed MySQL instance.

The concurrency profile is similarly undemanding, with at most a few simultaneous users. The computational requirements are limited to arithmetic aggregation, performed client-side after a single bulk load, which places no meaningful load on either the browser or the server.

All required technologies are mature, extensively documented and freely licensed. React, Vite, Node.js, Express and MySQL are among the most widely deployed components in contemporary web development, each with substantial community support. The development team possessed prior working knowledge of JavaScript, React and relational database design. The project was therefore assessed as clearly technically feasible.

2.6.2 Economic Feasibility

The economic case rests on the replacement of recurring manual effort by a small fixed infrastructure cost. Development was undertaken as an academic project and therefore incurred no labour cost to the business. The recurring operational cost consists of managed application hosting and a managed database instance, both of which are available at entry-level tiers adequate for this workload at a cost that is negligible relative to the value of the reconciliation time saved.

Against this, the quantifiable benefits are the reduction of daily reconciliation from between forty and seventy minutes to a few minutes of data entry that would in any case have been spent writing in a register; the elimination of stock and cash variances arising from arithmetic error; and the recovery of outstanding credit that was previously not systematically pursued because it was not systematically visible. The project was assessed as economically feasible with a payback period measured in weeks.

2.6.3 Operational Feasibility

Operational feasibility was the dimension of greatest concern, since the literature identifies organisational rejection rather than technical failure as the dominant cause of small-business software abandonment. Three mitigations were adopted. First, the daily entry screen was modelled on the section order of the existing physical register, so that the operator's working sequence is preserved rather than replaced. Second, no field was introduced that requires information the operator does not already possess at the moment of entry. Third, computed fields are displayed as read-only rather than being hidden, so that the operator can see and

sanity-check the arithmetic the system has performed on their behalf, which materially aided the transition of trust from paper to screen.

The system requires no specialised hardware, runs on the existing counter computer and on the proprietor's telephone, and requires no software installation. The project was assessed as operationally feasible subject to those design commitments, which were honoured.

2.6.4 Schedule Feasibility

The work was decomposed into eleven phases spanning approximately twenty-four weeks, as detailed in the timeline chart in Section 3.3. The schedule allocated the largest single blocks to backend endpoint development and to frontend component construction, with a deliberate five-week allocation to the business calculation engine in recognition that the stock and cash formulae represent the highest-risk element of the project. A four-week integration and testing phase preceded deployment. The schedule was assessed as feasible and was substantially met.

2.7 Requirement Specification

The requirements gathered during the analysis phase are set out below, separated into functional requirements describing what the system must do and non-functional requirements describing the qualities it must exhibit.

2.7.1 Functional Requirements

Table 2: Functional Requirements

ID	Requirement	Priority
FR-01	The system shall authenticate users against stored credentials and issue a time-limited session token on success.	High
FR-02	The system shall determine the user's role from the session token and present a role-appropriate interface.	High
FR-03	The system shall load the complete working dataset in a single request after authentication.	High
FR-04	The system shall allow recording of one daily entry per calendar date, identified uniquely by that date.	High
FR-05	The system shall restrict non-administrative users to recording the current working date only.	High
FR-06	The system shall allow administrators to open, edit and delete entries for any historical date.	High
FR-07	The system shall capture opening cash and bank deposit for each day.	High
FR-08	The system shall capture, per product, the cash sale, online sale, SBC and DBC quantities together with their applicable rates.	High
FR-09	The system shall automatically derive closing full-cylinder stock from opening stock,	High

ID	Requirement	Priority
	arrivals and all outward movements.	
FR-10	The system shall carry the derived closing stock forward as the following day's opening stock.	High
FR-11	The system shall record whether a cylinder vehicle arrived and, if so, filled cylinders received and empty cylinders despatched per product.	High
FR-12	The system shall maintain a running empty-cylinder balance netting all receipts against all plant despatches.	High
FR-13	The system shall capture physically counted godown stock of filled and empty cylinders per product per day.	Medium
FR-14	The system shall allow an informational shortage or stolen quantity to be recorded per product per day.	Medium
FR-15	The system shall record the cash and online split of cylinders delivered by each named delivery employee.	High
FR-16	The system shall capture an itemised list of sundry expenses per day.	High
FR-17	The system shall capture cheque and online receipts per day.	Medium
FR-18	The system shall capture accessory sales with quantity and rate.	Medium
FR-19	The system shall capture sundry cash credits with description and amount.	Medium
FR-20	The system shall record credit sales identifying customer, product, filled and empty cylinder quantities, amount and remarks.	High
FR-21	The system shall record recoveries against credit sales including any empty cylinders returned.	High
FR-22	The system shall automatically mark a credit ledger row as cleared once cumulative recoveries meet or exceed the original amount.	High
FR-23	The system shall prevent deletion of a credit ledger row that carries one or more recorded payments.	High
FR-24	The system shall capture vehicle running expenses categorised by type and attributed to a vehicle in the fleet master.	Medium
FR-25	The system shall capture salary and advance payments attributed to an employee and to a salary month.	High
FR-26	The system shall derive closing cash on hand from opening cash, all receipts and all disbursements.	High
FR-27	The system shall persist a complete daily entry as a single atomic database transaction.	High
FR-28	The system shall maintain effective-dated product price history including SBC and DBC rates.	High
FR-29	The system shall maintain effective-dated distributor commission history per product.	High
FR-30	The system shall provide create, read, update, activate, deactivate and delete operations for the vehicle master.	Medium
FR-31	The system shall provide create, read, update, activate, deactivate and delete operations for the employee master.	Medium
FR-32	The system shall allow administrators to create and remove system user accounts.	Medium

ID	Requirement	Priority
FR-33	The system shall present a management dashboard with month-to-date sales, month-on-month comparison, cylinder throughput, commission earned and total outstanding credit.	High
FR-34	The system shall present a consolidated history of all recorded days with drill-down to an individual day report.	High
FR-35	The system shall present a period summary supporting weekly, monthly and all-time aggregation.	Medium
FR-36	The system shall present a salary report showing base salary, advances drawn, salary paid and balance per employee for the applicable salary month.	Medium
FR-37	The system shall present a consolidated credit ledger showing every credit sale with its recovery history and outstanding balance.	High
FR-38	The system shall notify the user and require re-authentication when the session token expires.	High

2.7.2 Non-Functional Requirements

Table 3: Non-Functional Requirements

Category	Requirement
Performance	The consolidated master load shall complete within three seconds over an ordinary mobile-broadband connection. A daily entry save shall commit within two seconds. Navigation between tabs after load shall be instantaneous, as all data is served from client memory.
Reliability	A daily entry save shall be atomic; a failure at any step shall leave the database exactly as it was before the request. Schema migrations shall be idempotent and shall not fail a deployment if already applied.
Integrity	Referential integrity shall be enforced by foreign key constraints. Financial records that carry dependent history shall be protected from cascading deletion.
Security	All traffic shall be carried over HTTPS. Database connections shall use SSL. All protected endpoints shall require a valid bearer token. Sessions shall expire automatically after one hour of validity. All SQL shall use parameterised prepared statements.
Usability	The daily entry screen shall follow the section order of the agency's existing physical register. Computed values shall be visible to the operator rather than hidden. Destructive actions shall require explicit confirmation. The interface shall remain usable on a mobile viewport.
Portability	The application shall run in any current standards-compliant browser without installation, on desktop and mobile devices alike.
Maintainability	Business calculation logic shall be centralised in a single module rather than duplicated across components. Presentation tokens shall be centralised. Schema evolution shall be expressed in code and applied automatically.
Scalability	The architecture shall permit horizontal scaling of the application tier without shared session state, and the connection pool shall bound concurrent database usage.
Availability	The system shall be hosted on managed infrastructure providing automated restart and provider-level database backup.

Chapter 3: Design — Analysis, Design Methodology

The design methodology adopted for this system was deliberately conservative. The business it serves cannot tolerate data loss or silent corruption, and the development window was fixed. Architectural novelty was therefore treated as a risk rather than a virtue, and every significant design decision was resolved in favour of the option whose failure modes were best understood.

The system follows a classical three-tier layered architecture with a clear separation of presentation, application logic and persistence. Within the presentation tier, a component-based design is used with a single global state container; within the application tier, a route-handler organisation is used with all transaction boundaries held at the handler level; and within the data tier, a normalised relational schema is used with referential integrity enforced by the database rather than by application convention.

3.1 Hardware and Software Requirements

Because the system is delivered as a browser-based application, the hardware requirements imposed on the end user are minimal and are satisfied by any equipment already present at a working retail counter. There is no requirement for specialised peripherals, barcode readers, biometric devices or dedicated servers on the premises.

3.1.1 Client-Side Requirements

Table 4: Client-Side Hardware Requirements

Component	Minimum	Recommended
Processor	Any dual-core processor from the last decade	Quad-core desktop or contemporary mobile processor
Memory	2 GB RAM	4 GB RAM or more
Display	1024 × 768 resolution	1366 × 768 or higher; the layout also adapts to mobile viewports
Storage	Negligible; no local installation is performed	Negligible
Network	Any broadband or mobile data connection capable of sustaining 1 Mbps	5 Mbps or better for the fastest initial load
Input	Standard keyboard and pointing device, or touch screen	Numeric keypad accelerates daily entry considerably
Browser	Google Chrome, Microsoft Edge, Mozilla Firefox or Safari, current major version	Latest stable Chrome or Edge

The client requirement is deliberately undemanding because it is a hard constraint of the deployment context: the system had to run on the computer already present at the counter, which is an ordinary commodity desktop, and on the proprietor's existing mobile handset. No hardware purchase was contemplated as part of this project.

3.1.2 Server-Side and Development Requirements

Table 5: Server-Side and Development Environment Requirements

Component	Specification
Application host	Managed container platform providing a Node.js runtime with HTTPS termination, automatic restart and environment-variable configuration
Application runtime	Node.js 18 LTS or later
Application memory	512 MB allocated; observed working set well below this figure
Database host	Managed MySQL 8.x instance with SSL-enforced connections and automated backup
Database storage	1 GB provisioned; actual growth is a few megabytes per year for a single distributorship
Connection pooling	Ten concurrent connections, which comfortably exceeds the observed requirement
Development machine	Any workstation with 8 GB RAM running Node.js, a local MySQL instance and a modern code editor
Version control	Git, with the repository hosted remotely and deployment triggered from the main branch

3.1.3 Software Stack

Table 6: Software Stack and Version Matrix

Layer	Technology	Version	Role in the System
Presentation	React	19.2	Component model, declarative rendering and state management for the single-page application
Presentation	React DOM	19.2	Browser rendering target for the React component tree
Build	Vite	8.0	Development server with hot module replacement and optimised production bundling
Build	@vitejs/plugin-react	6.0	React fast-refresh integration for the Vite pipeline
UI	SweetAlert2	11.26	Modal confirmation and notification dialogs for save, delete and session-expiry events
Quality	ESLint	10.2	Static analysis with React hooks and refresh rule sets
Application	Node.js	18 LTS+	Server-side JavaScript runtime
Application	Express	5.2	HTTP routing, middleware pipeline and JSON body parsing

Layer	Technology	Version	Role in the System
Application	jsonwebtoken	9.0	Signing and verification of HS256 session tokens
Application	cors	2.8	Cross-origin resource sharing headers for the separately hosted frontend
Data access	mysql2	3.22	Promise-based MySQL driver with connection pooling and prepared statements
Data	MySQL	8.x	Relational database engine with InnoDB transactional storage

Two aspects of this stack merit explanation. First, the application deliberately carries no state-management library, no component framework and no charting dependency beyond what is listed. The dependency surface was kept small because every additional library is a maintenance and security obligation, and for an application of this size the built-in facilities of React proved entirely sufficient. Second, the mysql2 driver was selected over alternatives specifically for its promise interface, its native connection pooling and its support for server-side prepared statements, which together provide both the transactional control and the injection resistance the system requires.

3.2 Program / Module Specification

The system is decomposed into cohesive modules with deliberately narrow interfaces between them, following the principles of high cohesion and low coupling. The decomposition is described below first for the presentation tier and then for the application tier.

3.2.1 Presentation Tier Modules

The frontend is organised as a small number of substantial modules rather than a large number of fragments, reflecting the fact that the business screens are themselves large and that splitting them would have introduced prop-drilling without improving comprehensibility.

Table 7: Frontend Module Specification

Module	Responsibility
App.jsx	The global state container and application shell. Holds the authoritative client-side copies of entries, pending credits, prices, commissions, delivery boys, employees and vehicles, together with the currently open daily entry. Performs startup session verification, orchestrates the consolidated master load, owns the save, delete and payment-recording handlers, renders the header and tab navigation, and dispatches to the appropriate screen component according to the selected tab and the authenticated role.
LoginScreen.jsx	The unauthenticated entry point. Collects username and password, invokes the authentication endpoint, surfaces validation and server errors inline, and on success passes the issued token and role upward to the shell.
UserSide.jsx	The five operator-facing screens. DailyEntry renders the complete data-capture form

Module	Responsibility
	with all sixteen sections and every derived read-only field. History renders the consolidated day register with drill-down. PendingCredits lists uncleared ledger rows and records recoveries. Summary provides weekly, monthly and all-time aggregation. SalaryReport shows the per-employee monthly position. A GodownStock helper renders the physical-count section.
AdminSide.jsx	The eleven administrator-facing screens. AdminDashboard renders the key performance indicators and product snapshot. AdminPriceHistory and AdminCommission manage the effective-dated rate tables. AdminDayReports and AdminDayDetail render per-day reporting. AdminSalaryReport provides the administrative payroll view. AdminCreditOverview renders the consolidated ledger. AdminUsers, AdminVehicleMaster and AdminEmployeeMaster provide master-data management.
api.js	The service layer. Provides a single typed surface over all twenty-four endpoints, injects the bearer token into every authenticated request, intercepts authentication failures centrally, clears the stored credential and raises a session-expiry event that the shell handles, and appends cache-busting parameters to read requests.
constants.js	The business calculation engine and domain catalogue. Defines the product and accessory catalogues, the effective-dated rate resolvers for standard, SBC, DBC and commission rates, the date and currency formatters, the salary-month rule, the complete daily-entry calculation function, and the blank-template factories that construct a new day's entry with the correct carried-forward opening figures.
styles.js	The centralised design system. Exports the colour, typography and spacing tokens and injects the application stylesheet, ensuring visual consistency without a CSS framework dependency.

3.2.2 Application Tier Modules

The backend is organised as a single Express application with a middleware pipeline, a migration routine that executes before the listener is bound, and a set of route handlers grouped by resource. The complete endpoint surface is specified below.

Table 8: Backend REST API Endpoint Specification

Method	Path	Auth	Description
POST	/api/login	Public	Validates credentials and returns a signed HS256 token with a one-hour expiry together with the user's role
POST	/api/verify-session	Public	Validates a supplied token and returns its role, username and expiry claim; used at application startup
GET	/api/load	Token	Returns the consolidated master dataset: price history, commission history, delivery boys, employees, active vehicles, the full credit ledger with derived recovery totals, and every daily entry with all dependent collections
POST	/api/entries	Token	Persists a complete daily entry as a single SQL transaction across twelve tables
DELETE	/api/entries/:date	Token	Removes the operational records for a date while preserving permanent financial records

Method	Path	Auth	Description
POST	/api/payments	Token	Records a recovery against a credit ledger row and clears the row when cumulative recoveries meet the original amount
POST	/api/prices/sync	Token	Replaces the effective-dated price history within a transaction
POST	/api/commissions/sync	Token	Replaces the effective-dated commission history within a transaction
GET	/api/users	Token	Lists system user accounts
POST	/api/users	Token	Creates a system user account
DELETE	/api/users/:id	Token	Removes a system user account
GET	/api/vehicles	Token	Lists all vehicles in fleet order
POST	/api/vehicles	Token	Adds a vehicle to the fleet master
PUT	/api/vehicles/:id	Token	Updates a vehicle record
PATCH	/api/vehicles/:id/toggle	Token	Activates or deactivates a vehicle
DELETE	/api/vehicles/:id	Token	Removes a vehicle record
GET	/api/employees	Token	Lists all employees with role, salary, contact and joining date
POST	/api/employees	Token	Adds an employee to the master
PUT	/api/employees/:id	Token	Updates an employee record
PATCH	/api/employees/:id/toggle	Token	Activates or deactivates an employee
DELETE	/api/employees/:id	Token	Removes an employee record
GET	/api/godown-stock/:date	Token	Returns the physically counted godown stock for a date
POST	/api/godown-stock	Token	Upserts the physically counted godown stock for a date

3.2.3 The Middleware Pipeline

Every request entering the application tier passes through an ordered pipeline before reaching a handler. Cross-origin headers are applied first, permitting the separately hosted frontend to invoke the interface. The JSON body parser is applied next, with an elevated size limit to accommodate the full daily entry payload, which carries sixteen collections and can be substantial. The token verification middleware is then applied selectively to protected routes; it extracts the bearer credential, verifies the signature and expiry, attaches the decoded claims to the request, and returns a distinct response for an expired token so that the client can present an accurate message to the user.

3.2.4 The Transaction Manager

The daily-entry handler is the most consequential piece of server-side logic in the system, and its structure warrants explicit description. A connection is acquired from the pool and a transaction is opened. Twelve persistence steps then execute in sequence, in an order chosen so that dependencies are satisfied: the entry header is upserted first, so that dependent rows have a parent; the product stock rows are replaced; the delivery rows are replaced after resolving delivery-boy names against the employee master; the expense, cheque, vehicle-expense, salary, accessory and sundry-credit collections are replaced; the credit ledger is reconciled; and the godown and arrival records are upserted last. If every step succeeds the transaction commits; if any step raises, the transaction is rolled back in full and the error is returned to the client. In either case the connection is returned to the pool in a finally block, so that a failure cannot leak a connection and exhaust the pool.

3.2.5 The Migration Runner

Schema evolution is handled by a routine that executes once at server start-up, before the HTTP listener is bound. Each migration first interrogates the live schema to determine whether it has already been applied, and applies its change only if required. Every migration is wrapped so that a failure is logged as a warning rather than aborting start-up, on the principle that a server which starts with a slightly stale schema is preferable to a server which does not start at all. This design allows a new application version to be deployed against an existing database without any manual administration step, and allows the same deployment to be repeated safely.

3.3 Timeline Chart

The project was executed over approximately twenty-four weeks, decomposed into eleven overlapping phases. The allocation deliberately front-loads analysis and reserves an extended block for the business calculation engine, in recognition that the stock and cash formulae — rather than any framework-level concern — represented the principal technical risk of the project.

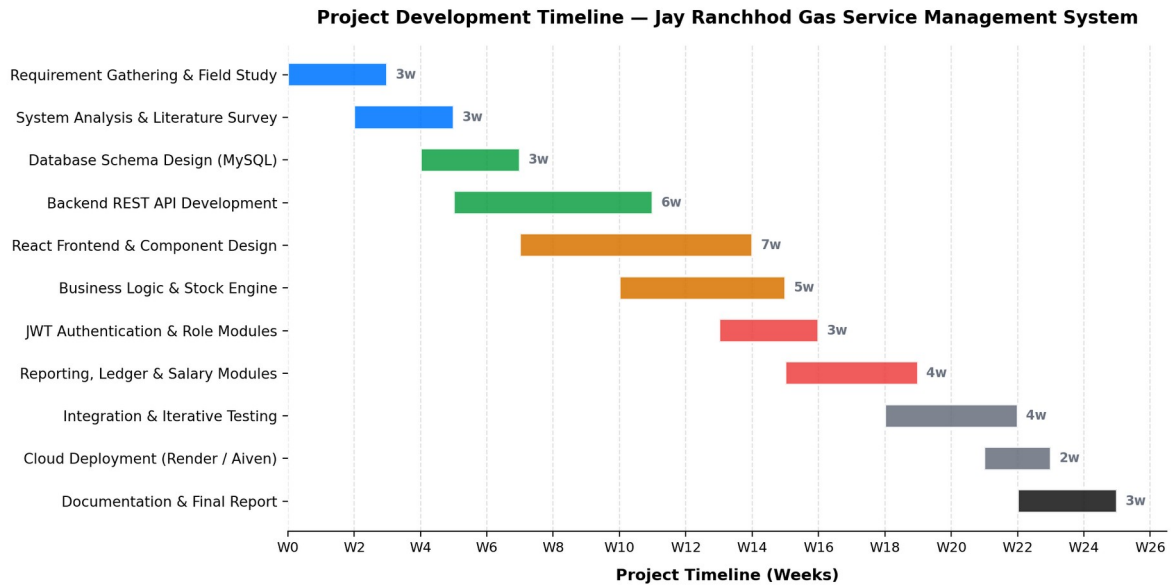


Fig 1: Project Development Timeline (Gantt Chart)

Table 9: Project Phase Breakdown

Phase	Duration	Principal Deliverables
Requirement Gathering & Field Study	3 weeks	Counter observation, staff interviews, examination of existing registers, documented working sequence
System Analysis & Literature Survey	3 weeks	Functional and non-functional requirement specification, feasibility assessment, literature review
Database Schema Design	3 weeks	Entity relationship model, nineteen-table normalised schema, referential integrity constraints
Backend REST API Development	6 weeks	Express application, middleware pipeline, twenty-four endpoints, transaction manager, migration runner
React Frontend & Component Design	7 weeks	Application shell, service layer, design system, all sixteen screens across both roles
Business Logic & Stock Engine	5 weeks	Closing stock derivation, In-Out Stock Master, cash-on-hand formula, effective-dated rate resolution
JWT Authentication & Role Modules	3 weeks	Token issuance and verification, startup session validation, expiry interception, role-based interface partitioning
Reporting, Ledger & Salary Modules	4 weeks	Dashboard, day reports, period summary, salary report, consolidated credit ledger
Integration & Iterative Testing	4 weeks	Unit and integration test execution, parallel running against the manual register, defect resolution
Cloud Deployment	2 weeks	Managed application host configuration, managed database provisioning, SSL, environment configuration
Documentation & Final Report	3 weeks	System documentation, diagrams, this report

3.4 UML and Structured Design Diagrams

The logical complexity of the application — in particular the interaction between the client-side calculation engine, the transactional server, the effective-dated rate tables and the chain-dependent inventory model — required systematic visual modelling before implementation. Structured analysis diagrams and Unified Modelling Language diagrams were prepared during the design phase and were maintained as the design evolved. They are presented in the following sections.

3.4.1 Flowchart Diagram

The flowchart below traces the two principal operational sequences of the system. Phase A covers the establishment of a session: the application first checks for a stored credential, verifies it against the server if present, and otherwise presents the login screen; on successful authentication a signed token with a one-hour lifetime is issued, the role claim is decoded, the corresponding portal is mounted, and the consolidated master dataset is retrieved. Phase B covers the capture and persistence of a day's trading: the working date is selected, vehicle arrivals are recorded if applicable, the sixteen data-capture sections are completed, the derived stock and cash figures are computed and displayed, and on save the server executes the twelve-step transaction with commit or rollback as its outcome.

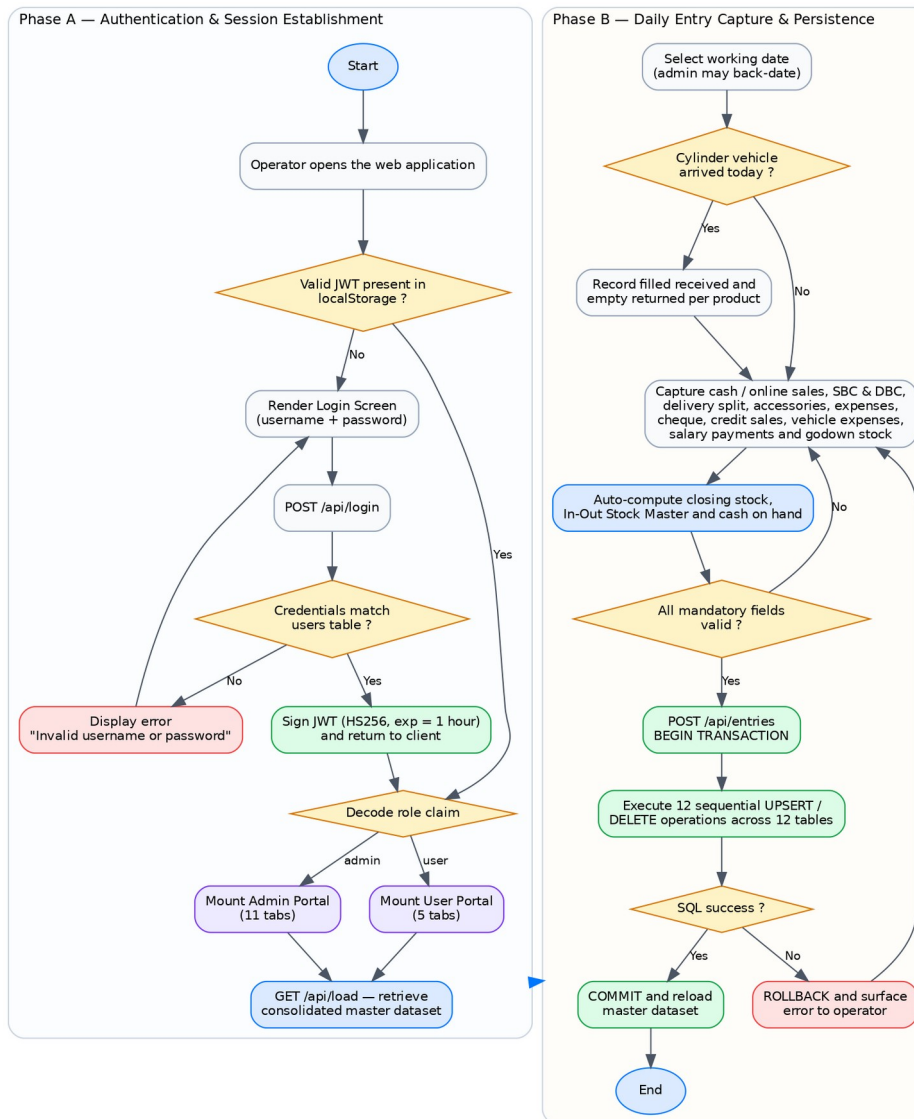


Fig 2: Flowchart Diagram — Authentication and Daily Entry Lifecycle

3.4.2 Use Case Diagram

The use case model identifies two human actors and one system actor. The counter operator exercises the seven use cases concerned with routine daily operation. The administrator exercises those same seven — the administrative interface is a superset, not an alternative — together with ten further use cases covering back-dated correction, master-data management, rate administration, reporting and user administration. The database participates as a supporting actor in the persistence and retrieval use cases.

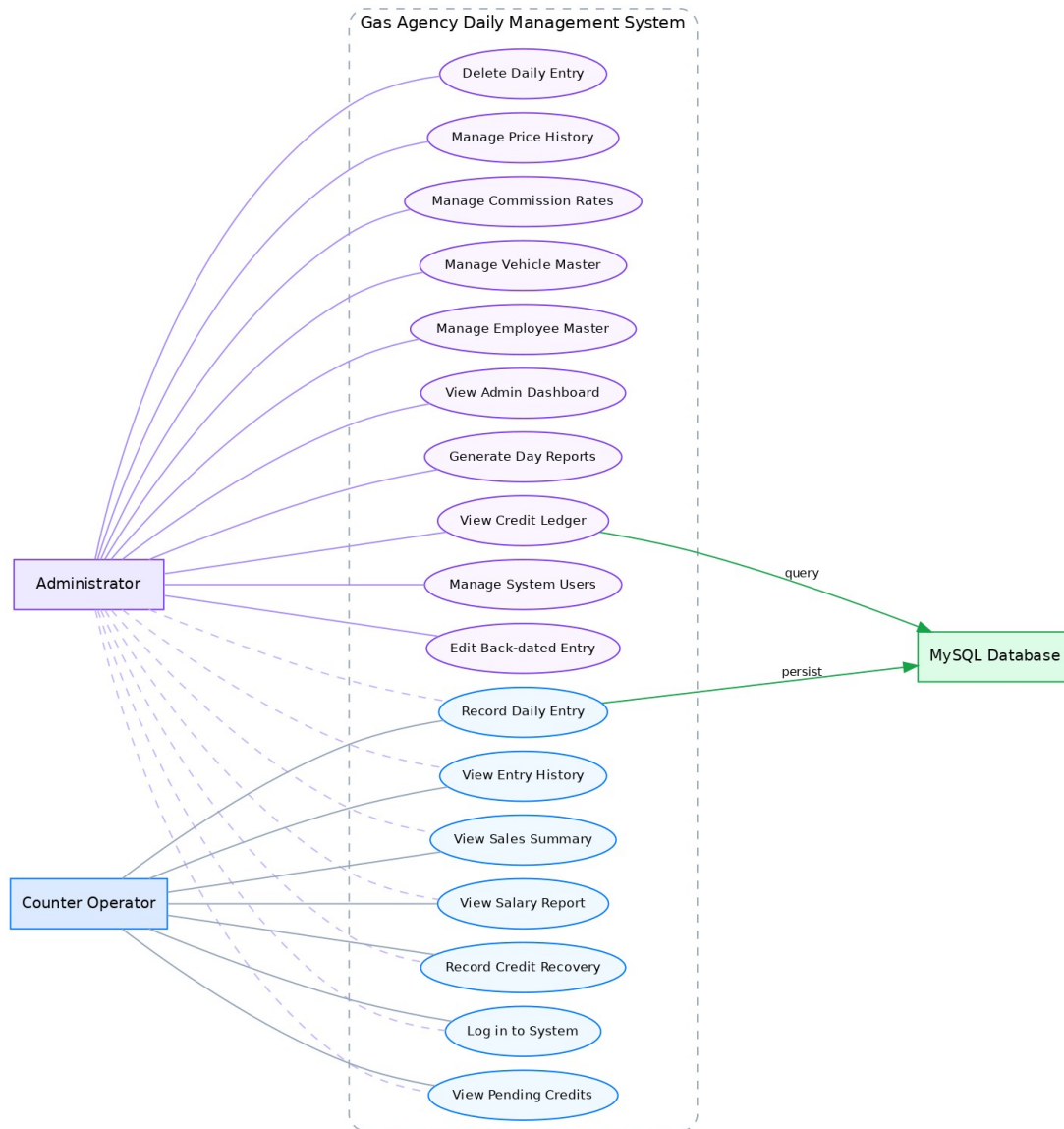


Fig 3: Use Case Diagram

3.4.3 Level 0 Data Flow Diagram

The context diagram establishes the boundary of the system and its four external entities. The counter operator supplies credentials and the day's operational data and receives the derived cash, stock and credit positions. The administrator supplies rate and master configuration and receives management reporting. The credit customer is the source of credit purchases and recovery payments and the recipient of balance information. The bottling plant is the source of filled cylinders and the destination of empty cylinders, and it is the inclusion of this fourth entity that distinguishes an LPG distribution model from an ordinary retail inventory model.

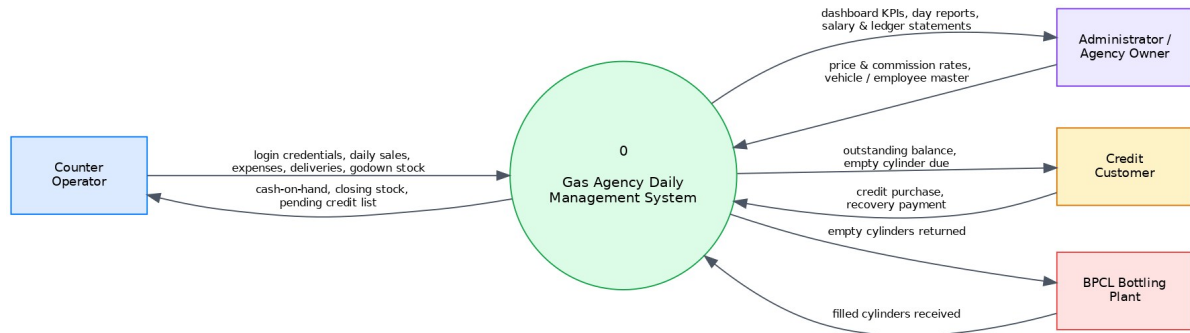


Fig 4: Level 0 Data Flow Diagram (Context Diagram)

3.4.4 Level 1 Data Flow Diagram

Decomposing the single context process yields six major processes and six logical data stores. Process 1.0 authenticates users against the user store and issues tokens. Process 2.0 captures the daily entry and writes to the daily transaction and stock stores. Process 3.0 computes the derived stock and cash figures and writes the closing stock back. Process 4.0 manages the credit ledger and its payment history. Process 5.0 maintains the rate and master stores. Process 6.0 reads across the stores to generate management reporting.

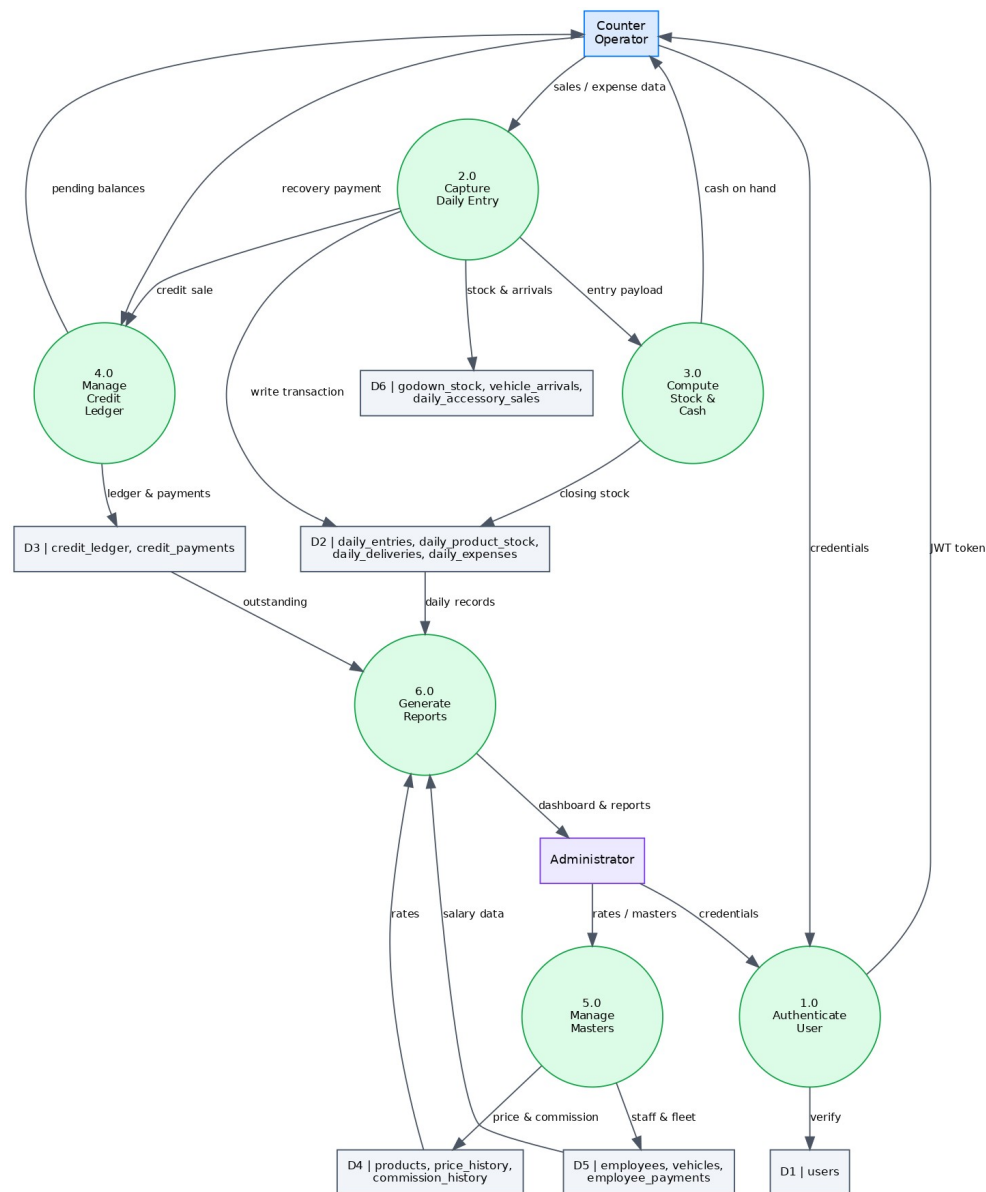


Fig 5: Level 1 Data Flow Diagram

3.4.5 Entity Relationship Diagram

The entity relationship model comprises nineteen tables organised into four functional groups. Four master entities — users, products, employees and vehicles — hold reference data. Two effective-dated history entities hold prices and commissions. The `daily_entries` entity acts as the parent of ten dependent daily transaction entities, keyed by trading date. Two stock entities hold the physical godown count and the plant arrival record. Two credit entities hold the ledger and its payment history in a one-to-many relationship that is the reason the credit ledger cannot be persisted by naive replacement.

Note that `daily_entries` is keyed by the trading date itself rather than by a surrogate identifier. This is a deliberate modelling decision: a trading date is a natural, immutable, unique business

key, and using it directly makes the one-entry-per-day invariant a property of the schema rather than a rule the application must remember to enforce.

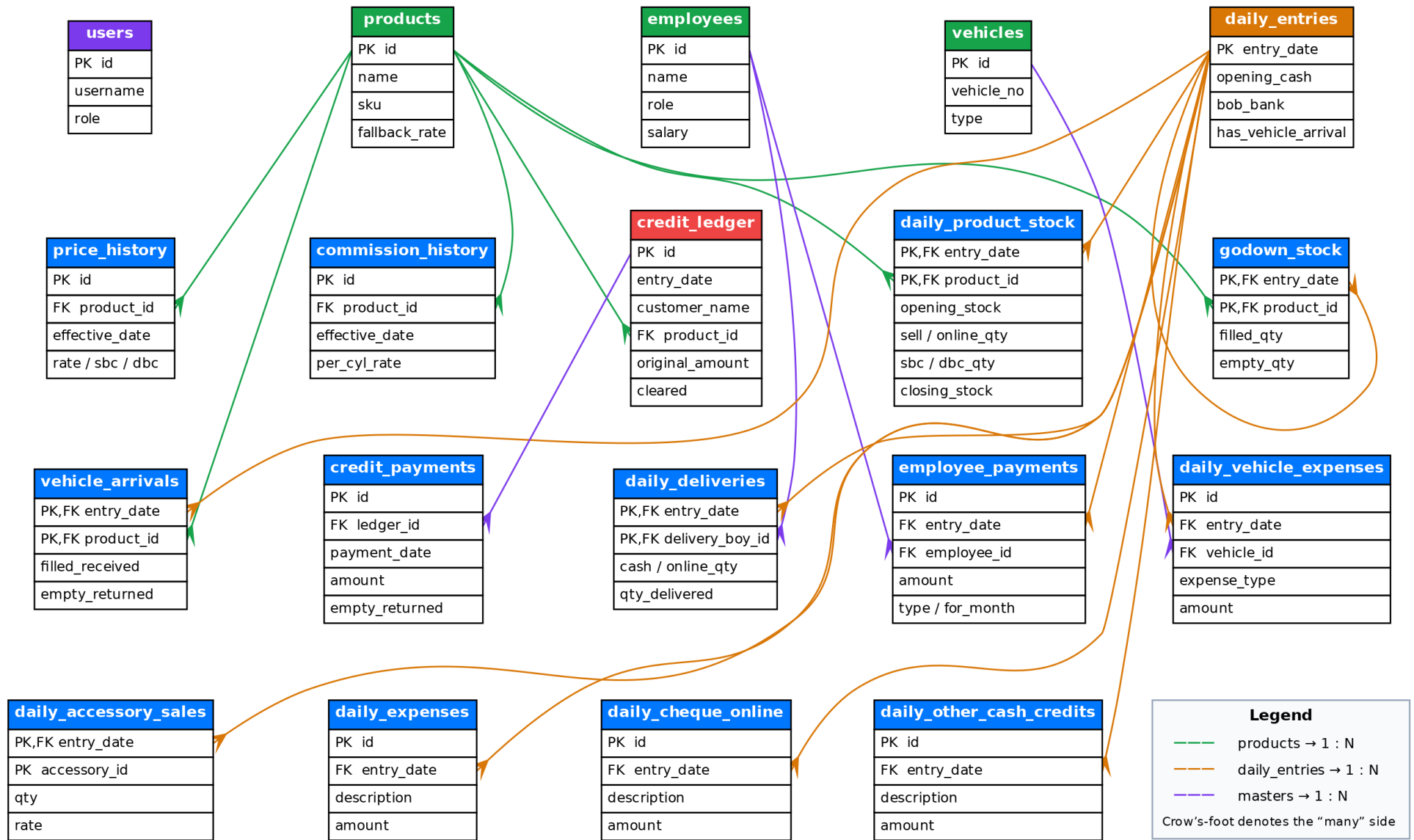


Fig 6: Entity Relationship Diagram

3.4.6 Sequence Diagram

The sequence diagram traces a complete operational cycle across six participants. Messages one to seven cover authentication and token issuance. Messages eight to fourteen cover the consolidated master load, showing the token verification middleware intercepting the request before the handler executes and the handler issuing fifteen parallel queries to assemble the response. Messages fifteen to nineteen cover the capture and transactional persistence of a daily entry, concluding with the confirmation returned to the operator.

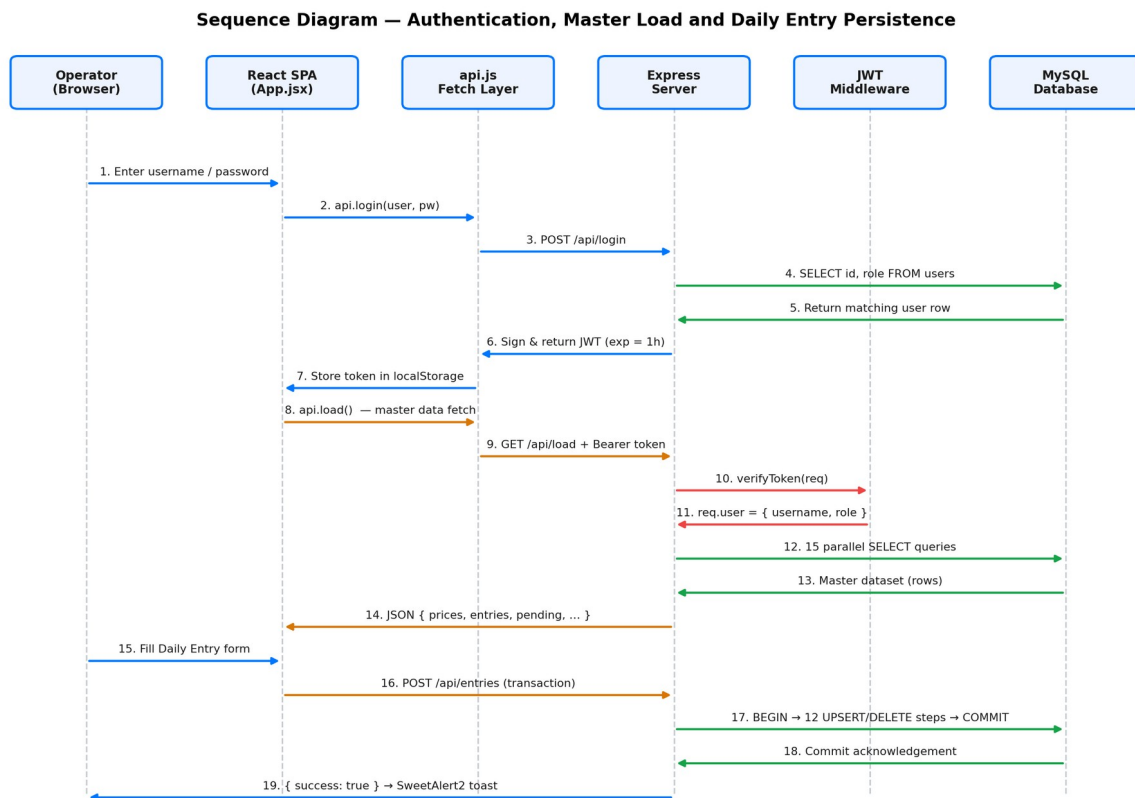


Fig 7: Sequence Diagram — Login, Master Load and Entry Persistence

Chapter 4: Implementation

4.1 System Flow

The implemented system operates as three physically separated tiers communicating over well-defined interfaces. This section describes the runtime flow through those tiers, from the moment the operator opens the application to the moment a day's trading is durably committed.

4.1.1 Architectural Overview

The presentation tier is delivered as a static bundle produced by the build tool and served over HTTPS. On load it mounts the application shell, which immediately attempts to validate any credential held in browser storage. The application tier runs as a long-lived Node process on a managed container platform, exposing the interface over HTTPS with the platform terminating transport security. The data tier is a managed MySQL instance reachable only over an SSL-enforced connection, accessed exclusively through a bounded connection pool held by the application tier.

No component of the presentation tier ever addresses the database directly, and no credential for the database exists anywhere in the client bundle. This is the fundamental security property of the arrangement: the browser is trusted to render and to collect input, and is trusted with nothing else.

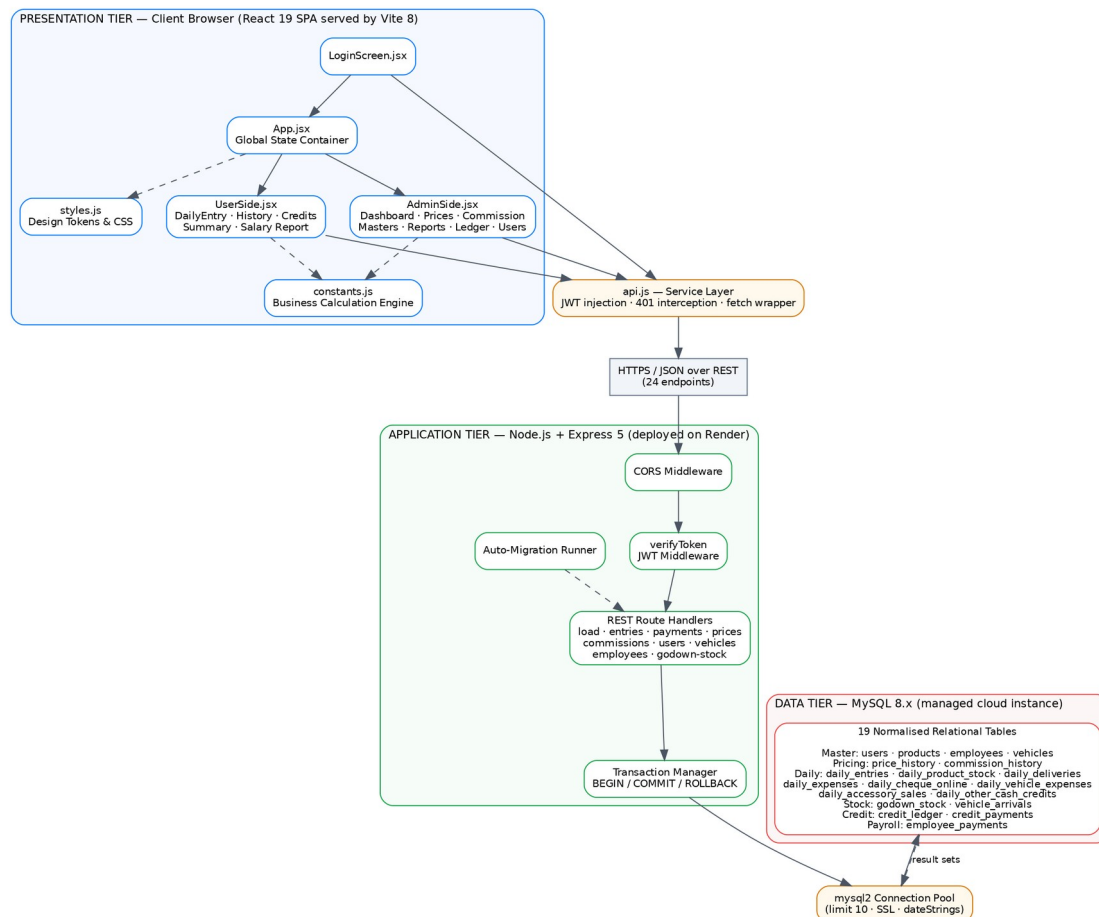


Fig 8: System Flow / Three-Tier Architecture Diagram

4.1.2 Session Establishment

When the application mounts, it reads any token held in browser storage and submits it to the verification endpoint. This deliberate round trip exists because a token in local storage cannot be trusted on its face: it may have expired, it may have been issued by a previous deployment carrying a different signing key, or it may have been tampered with. Only the server can determine its validity. If verification succeeds, the returned role determines which portal is mounted and the master load proceeds immediately. If verification fails, the stored credential is discarded and the login screen is presented.

Authentication itself submits the supplied credentials to the login endpoint, which resolves them against the user table using a parameterised statement. On success the server constructs a claim set containing the username and role, signs it with the HS256 algorithm and an expiry one hour in the future, and returns the token together with the role. The client stores the token and passes the role upward to the shell, which selects the tab set and triggers the master load.

4.1.3 The Consolidated Master Load

Rather than fetching data screen by screen, the application retrieves its entire working dataset in a single request immediately after authentication. The handler for this endpoint issues fifteen queries covering products, price history, commission history, active employees, active vehicles, the credit ledger, the credit payment history, the daily entry headers, and the eight dependent daily collections, together with the godown, arrival and accessory records.

The handler then performs the assembly in application code rather than in SQL. Credit ledger rows are joined with their payments to derive a recovered total and a clearance flag. Daily entry headers are joined with their dependent collections, keyed by trading date, to produce fully populated entry objects. Recovery payments are additionally projected onto the date on which they were received, so that a payment received today appears within today's entry even though it belongs to a ledger row created weeks earlier.

This design was chosen over per-screen fetching for a specific operational reason. The counter's connection is mobile broadband with high and variable latency, under which an interface that issues a request on every navigation feels unusably slow. By paying the latency cost once, the application makes every subsequent screen transition instantaneous, which materially affects whether the staff are willing to use it.

4.1.4 Daily Entry Capture and Persistence

The daily entry screen presents sixteen sections following the order of the agency's physical register. As the operator types, the calculation engine recomputes the derived values on every change and displays them as read-only fields, so that the closing stock and cash position are always visible and always current.

On save, the shell first finalises the entry by computing the definitive closing stock for each product from the opening stock, the arrivals and every outward movement category. The completed payload is then submitted to the persistence endpoint, which executes the twelve-step transaction described in Section 3.2.4. On a successful commit the client reloads the master dataset and restores the view to the date that was saved — a detail that matters when an administrator is editing a back-dated entry, since a naive reload would return the view to today and lose the operator's place.

4.2 Database Design

The database comprises nineteen tables in third normal form, using the InnoDB storage engine for transactional integrity and foreign key enforcement. The design principles applied were:

use natural business keys where a genuine natural key exists; enforce referential integrity in the schema rather than in application code; store rates as effective-dated history rather than as mutable current values; and never store a value that can be reliably derived, with the single deliberate exception of the closing stock, which is stored because it must be carried forward as the next day's opening figure.

4.2.1 Table Inventory

Table 10: Database Table Inventory by Functional Group

Functional Group	Tables	Purpose
Master / Reference	users, products, employees, vehicles	Identity, product catalogue, staff register and fleet register
Effective-Dated Rates	price_history, commission_history	Date-stamped product rates including SBC and DBC channels, and distributor commission per cylinder
Daily Transaction	daily_entries, daily_product_stock, daily_deliveries, daily_expenses, daily_cheque_online, daily_vehicle_expenses, daily_accessory_sales, daily_other_cash_credits	The trading-day header and its seven dependent collections
Stock Movement	godown_stock, vehicle_arrivals	Physically counted stock and plant receipt / despatch records
Credit	credit_ledger, credit_payments	Customer credit sales and their recovery history
Payroll	employee_payments	Salary and advance payments attributed to employee and salary month

4.2.2 Data Dictionary — Master and Reference Tables

Table 11: Data Dictionary — Master and Reference Tables

Table	Column	Type	Constraint / Description
users	id	INT	Primary key, auto-increment
	username	VARCHAR(50)	Unique login identifier
	password	VARCHAR(255)	Authentication secret
	role	ENUM	Either 'admin' or 'user'; determines the interface presented
products	id	VARCHAR(10)	Primary key; 'p14', 'p19' or 'p5'
	name	VARCHAR(50)	Display name of the cylinder variant
	sku	VARCHAR(30)	Bharat Gas stock keeping code

Table	Column	Type	Constraint / Description
	fallback_rate	DECIMAL(10,2)	Rate used when no price history row exists
	fallback_sbc	DECIMAL(10,2)	SBC scheme fallback rate
	fallback_dbc	DECIMAL(10,2)	DBC scheme fallback rate
employees	id	INT	Primary key, auto-increment
	name	VARCHAR(100)	Employee name; delivery boys are matched by this value
	role	VARCHAR(50)	'Delivery Boy', 'Counter Staff', 'Driver' etc.
	salary	DECIMAL(10,2)	Monthly base salary used as the payroll entitlement
	phone	VARCHAR(20)	Contact number
	join_date	DATE	Date of joining
	notes	VARCHAR(255)	Free-text remarks
	is_active	BOOLEAN	Inactive employees are excluded from dropdowns but retained for history
vehicles	id	INT	Primary key, auto-increment
	vehicle_no	VARCHAR(20)	Registration number, normalised to upper case
	type	VARCHAR(30)	'3-Wheeler', 'Truck', 'Pickup' etc.
	capacity	INT	Cylinder carrying capacity
	notes	VARCHAR(255)	Free-text remarks
	is_active	BOOLEAN	Retired vehicles are deactivated rather than deleted
	sort_order	INT	Display ordering within the fleet list
price_history	id	VARCHAR(20)	Primary key
	product_id	VARCHAR(10)	Foreign key to products
	effective_date	DATE	Date from which this rate applies
	rate	DECIMAL(10,2)	Standard retail rate per cylinder
	sbc_rate	DECIMAL(10,2)	Rate for the SBC scheme channel
	dbc_rate	DECIMAL(10,2)	Rate for the DBC scheme channel
	note	VARCHAR(255)	Reason for the revision
commission_history	id	VARCHAR(20)	Primary key

Table	Column	Type	Constraint / Description
	product_id	VARCHAR(10)	Foreign key to products
	effective_date	DATE	Date from which this commission applies
	per_cyl_rate	DECIMAL(10,2)	Distributor commission per cylinder
	note	VARCHAR(255)	Reason for the revision

4.2.3 Data Dictionary — Daily Transaction Tables

Table 12: Data Dictionary — Daily Transaction Tables

Table	Column	Type	Constraint / Description
daily_entries	entry_date	DATE	Primary key; the natural business key enforcing one entry per trading day
	opening_cash	DECIMAL(12,2)	Cash carried forward from the previous day
	bob_bank	DECIMAL(12,2)	Amount deposited to the bank during the day
	has_vehicle_arrival	BOOLEAN	Whether a plant vehicle arrived on this date
daily_product_stock	entry_date	DATE	Composite primary key part; foreign key to daily_entries
	product_id	VARCHAR(10)	Composite primary key part; foreign key to products
	opening_stock	INT	Filled cylinders held at start of day, carried from the previous closing
	rate	DECIMAL(10,2)	Standard rate applied on this date, snapshotted from price history
	sbc_rate	DECIMAL(10,2)	SBC rate applied on this date
	dbc_rate	DECIMAL(10,2)	DBC rate applied on this date
	sell_qty	INT	Cylinders sold against cash
	online_qty	INT	Cylinders sold against digital payment
	sbc_qty	INT	Cylinders issued under the SBC scheme
	dbc_qty	INT	Cylinders issued under the DBC scheme
	closing_stock	INT	Derived closing stock; becomes the next day's opening figure
	shortage_qty	INT	Informational shortage or stolen reminder; does not enter the stock calculation
	remarks	VARCHAR(255)	Free-text note for the product line

Table	Column	Type	Constraint / Description
daily_deliveries	entry_date	DATE	Composite primary key part; foreign key to daily_entries
	delivery_boy_id	INT	Composite primary key part; foreign key to employees with cascade delete
	cash_qty	INT	Cylinders delivered and settled in cash
	online_qty	INT	Cylinders delivered and settled digitally
	qty_delivered	INT	Total cylinders delivered by this person on this date
daily_expenses	id	VARCHAR(20)	Primary key
	entry_date	DATE	Foreign key to daily_entries
	description	VARCHAR(255)	Nature of the expense
	amount	DECIMAL(10, 2)	Amount disbursed
daily_cheque_online	id	VARCHAR(20)	Primary key
	entry_date	DATE	Foreign key to daily_entries
	description	VARCHAR(255)	Instrument or reference detail
	amount	DECIMAL(10, 2)	Amount received
daily_vehicle_expenses	id	VARCHAR(20)	Primary key
	entry_date	DATE	Foreign key to daily_entries
	vehicle_id	INT	Foreign key to vehicles; attributes the cost to a specific vehicle
	expense_type	VARCHAR(30)	Fuel, Repair, Maintenance, Toll / Tax, Washing or Other
	description	VARCHAR(255)	Detail of the expense
	amount	DECIMAL(10, 2)	Amount disbursed
daily_accessory_sales	entry_date	DATE	Composite primary key part; foreign key to daily_entries
	accessory_id	VARCHAR(20)	Composite primary key part; 'pipe' or 'stove'
	qty	INT	Units sold
	rate	DECIMAL(10, 2)	Unit rate applied
daily_other_cash_credit	id	VARCHAR(20)	Primary key

Table	Column	Type	Constraint / Description
s		)	
	entry_date	DATE	Foreign key to daily_entries, indexed
	description	VARCHAR(255)	Nature of the sundry cash receipt
	amount	DECIMAL(10, 2)	Amount received

4.2.4 Data Dictionary — Stock, Credit and Payroll Tables

Table 13: Data Dictionary — Stock, Credit and Payroll Tables

Table	Column	Type	Constraint / Description
godown_stock	entry_date	DATE	Composite primary key part
	product_id	VARCHAR(10)	Composite primary key part; foreign key to products
	filled_qty	INT	Physically counted filled cylinders at close of day
	empty_qty	INT	Physically counted empty cylinders at close of day
vehicle_arrivals	entry_date	DATE	Composite primary key part
	product_id	VARCHAR(10)	Composite primary key part; foreign key to products
	filled_received	INT	Filled cylinders received from the bottling plant
	empty_returned	INT	Empty cylinders despatched to the bottling plant
credit_ledger	id	VARCHAR(40)	Primary key, composed of the trading date and a generated suffix
	entry_date	DATE	Date on which the credit was extended
	customer_name	VARCHAR(100)	Customer name, normalised to upper case for reliable grouping
	original_amount	DECIMAL(10, 2)	Value of the credit sale
	cleared	BOOLEAN	Set automatically once cumulative recoveries meet the original amount
	product_id	VARCHAR(10)	Product taken on credit
	filled_qty	INT	Filled cylinders issued against this credit
	empty_qty	INT	Empty cylinders returned at the time of the credit sale
	remarks	VARCHAR(255)	Free-text note
credit_payments	id	VARCHAR(20)	Primary key

Table	Column	Type	Constraint / Description
	ledger_id	VARCHAR(40)	Foreign key to credit_ledger; the relationship that makes naive replacement unsafe
	payment_date	DATE	Date the recovery was received
	amount	DECIMAL(10, 2)	Amount recovered in this instalment
	empty_returned	INT	Empty cylinders returned alongside this payment
	note	VARCHAR(255)	Free-text note
employee_payments	id	VARCHAR(20)	Primary key
	entry_date	DATE	Date the payment was made
	employee_id	INT	Foreign key to employees
	amount	DECIMAL(10, 2)	Amount paid
	type	ENUM	'Salary' or 'Advance'
	for_month	VARCHAR(7)	Salary month in YYYY-MM form, which may differ from the payment date
	notes	VARCHAR(255)	Free-text note

4.2.5 Notable Design Decisions

- **Date as natural primary key.** The daily_entries table is keyed by entry_date rather than by a surrogate identifier, making the one-entry-per-day invariant a schema property. All dependent daily tables key on the same date, so a day's records are trivially locatable and the upsert-on-duplicate-key pattern works naturally.
- **Rate snapshotting.** Although rates are resolvable from price_history, the applicable rate is also written into daily_product_stock at the moment of entry. This deliberate denormalisation guarantees that a historical day's revenue can never be disturbed by a later correction to the rate table, and removes the need for a temporal join on every report.
- **Soft deletion on masters.** Employees and vehicles carry an is_active flag. Retiring a delivery boy removes them from the dropdown without orphaning the historical delivery rows that reference them, preserving the integrity of past reporting.
- **Composite ledger identifiers.** Credit ledger identifiers embed the trading date. This allows the application to determine, without a join, whether a recovery received today relates to a credit extended today, which is required for the same-day credit netting rule described in Section 4.3.
- **Protected deletion of financial records.** Deleting a daily entry removes the operational records for that date but deliberately preserves the credit ledger, since those rows carry

recovery history belonging to other dates. Similarly, a credit ledger row removed from the entry form is deleted only if it carries no payments.

- **Idempotent schema migration.** Every schema change is expressed as a guarded routine that inspects the live schema before acting, so that deployment requires no manual database administration and can be repeated safely.

4.3 The Business Calculation Engine

All derived business quantities are computed by a single centralised module rather than being scattered across the components that display them. This section documents the principal formulae, which collectively constitute the domain logic of the system.

Table 14: Product Catalogue and Scheme Codes

Code	Product	SKU	Description
p14	14 KG Domestic Cylinder	5350–5370	The standard household LPG cylinder and the highest-volume product
p19	19 KG Commercial Cylinder	5400	Commercial cylinder supplied to hotels, canteens and small industry
p5	FLT 5 KG Cylinder	FLT	Small portable cylinder for low-consumption and transient customers
pipe	Gas Pipe	—	Accessory sold alongside cylinders; not part of cylinder stock
stove	Gas Stove	—	Accessory sold alongside cylinders; not part of cylinder stock
SBC	Single Bottle Connection	—	Subsidised scheme channel carrying its own independent rate
DBC	Double Bottle Connection	—	Subsidised scheme channel carrying its own independent rate

4.3.1 Effective-Dated Rate Resolution

Rates are never read as a single current value. The resolver filters the price history for the requested product, orders the surviving rows by effective date descending, and returns the rate of the most recent row not later than the date being evaluated, falling back to the product's catalogue default only if no history row exists. Commission resolution follows the same pattern with an explicit upper date bound, so that a commission calculation for a past month uses the commission rate that was actually in force during that month.

```

getCurrentRate(productId, priceHistory, uptoDate):
  rows ← priceHistory where productId matches and effectiveDate ≤ uptoDate
  rows ← sort rows by effectiveDate descending
  return rows.isEmpty ? product.fallbackRate : rows[0].rate

```

4.3.2 Closing Full-Cylinder Stock

The closing stock of each product is the opening stock, increased by any filled cylinders received from the bottling plant, and reduced by every category of outward movement: cash sales, online sales, SBC scheme cylinders, DBC scheme cylinders, and filled cylinders issued against credit. The result is written to the database and becomes the following day's opening stock, so that the inventory chain is maintained by the application and never by hand.

```
closingStock(product, day):
  opening ← product.openingStock           // carried from previous day
  received ← day.hasArrival ? arrival.filledReceived : 0
  creditOut ←  $\Sigma$  filledQty of credit sales for this product on this day
  return opening + received
           - sellQty - onlineQty - sbcQty - dbcQty - creditOut
```

The shortage quantity recorded on the product line is explicitly excluded from this formula. It exists as a managerial reminder that a discrepancy was observed, and is surfaced in the interface and in reporting, but it does not silently adjust the inventory chain. Absorbing a shortage into the computed stock would conceal precisely the information the field exists to expose.

4.3.3 The In-Out Stock Master Empty-Cylinder Balance

The empty-cylinder position is the quantity that the manual system could not produce at all, and it is computed as a running total across the complete history of trading days rather than from a stored opening figure. For each product, the engine replays every entry preceding the day under evaluation in chronological order, accumulating the empties received and subtracting the empties despatched, and thereby derives the opening empty balance for the current day.

```
openingEmpty(product, today):
  running ← 0
  for each day d in history where d.date < today, ascending:
    emptyIn ← d.sellQty + d.onlineQty + d.sbcQty + d.dbcQty
              +  $\Sigma$  emptyQty of credit sales for this product
              +  $\Sigma$  emptyReturned of credit recoveries for this product
    emptyOut ← d.hasArrival ? d.arrival.emptyReturned : 0
    running ← running + emptyIn - emptyOut
  return running

closingEmpty(product, today):
  return openingEmpty(product, today)
         + emptyIn(today) - emptyOut(today)
```

Three sources contribute empties. An ordinary sale of any channel returns one empty per cylinder, because the customer exchanges their empty for a filled unit. A credit sale may return empties immediately even though payment is deferred, and this quantity is recorded separately on the ledger row. A credit recovery may be accompanied by empties returned late, and this quantity is recorded on the payment row. Netting all three against the plant despatch produces

the authoritative balance, which is displayed alongside the physically counted godown figure so that any divergence is immediately visible.

4.3.4 Cash on Hand

The cash position is derived from opening cash by adding every receipt category and subtracting every disbursement category. Online sales are added as part of total sales and then deducted, because although they constitute revenue they never enter the physical cash drawer; retaining both terms keeps the revenue figure and the cash figure independently correct rather than conflating them.

```
cashOnHand(day):  
    return openingCash  
        + totalSales          // cash sales + online sales  
        + accessorySales  
        + creditRecoveries  
        + otherCashCredits  
        - onlineSales          // digital receipts never enter the drawer  
        - expenses  
        - vehicleExpenses  
        - salaryPayments  
        - bankDeposit
```

4.3.5 Same-Day Credit Netting

A specific commercial situation required explicit handling. A customer may take a cylinder on credit in the morning and settle it the same afternoon. Recorded naively, that day would show both an outstanding credit and a recovery, overstating both figures. The engine therefore identifies recoveries whose ledger identifier carries the current trading date, and nets them against the day's credit total so that a same-day settlement reports as zero outstanding credit. The cash sales figure is deliberately left unadjusted, because the physical cash drawer did receive that money and the cash reconciliation must reflect physical reality.

4.3.6 The Salary Month Rule

Payroll in the agency is settled in arrears during the first days of the following month. A payment made on the fifth of a month is therefore, in the overwhelming majority of cases, settlement of the preceding month's salary. The engine encodes this convention: if the current date falls on or before the tenth of the month, the applicable salary month defaults to the previous month; otherwise it defaults to the current month. The operator may override this default on any individual payment, and the chosen month is stored on the payment row so that the report reflects the true entitlement period rather than the accident of the payment date.

4.4 Module Specification (Screenshots of Working Modules)

The following sections present the implemented interface. The application is organised as a persistent header and tab navigation with a single active screen beneath, and the tab set is determined by the authenticated role: five tabs for the counter operator and eleven for the administrator.

4.4.1 Authentication

The login screen is the sole unauthenticated surface. It carries the agency branding, collects a username and password, and reports authentication and connectivity failures inline rather than through a modal, so that the operator can correct and retry without dismissing a dialog. On success the issued token is placed in browser storage and the appropriate portal is mounted.

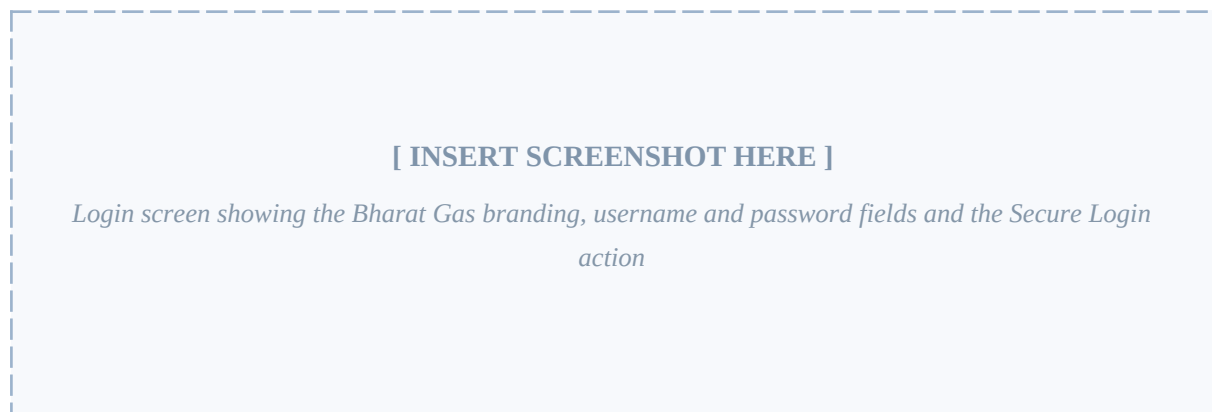


Fig 9: Login Screen

4.4.2 Daily Entry — Header, Vehicle Arrival and Product Stock

The daily entry screen opens with the working date, the opening cash carried forward from the previous day, and the bank deposit field. Beneath these sits the vehicle arrival question; answering affirmatively reveals a per-product table capturing filled cylinders received and empty cylinders despatched. The product stock section follows, presenting for each of the three cylinder variants the read-only opening stock and rate, the editable cash, online, SBC and DBC quantities, the automatically derived closing stock, the shortage reminder field and a remarks column. Running cash, online and total sales figures are displayed in the section header and update on every keystroke.

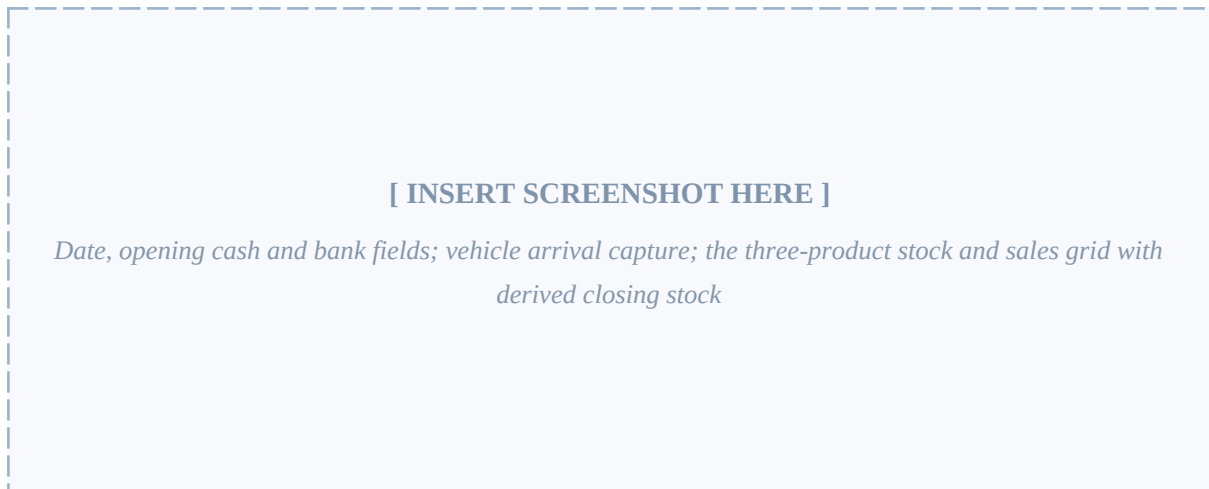


Fig 10: Daily Entry — Header, Vehicle Arrival and Product Stock

4.4.3 Daily Entry — Delivery Split and Accessories

The delivery section lists every active delivery employee and captures, for each, the cylinders delivered against cash and against digital payment. Recording the split rather than a single total is what permits the delivery figures to be reconciled against the cash drawer. The accessory section captures sales of gas pipes and stoves, each with a quantity and a rate defaulted from the price history.

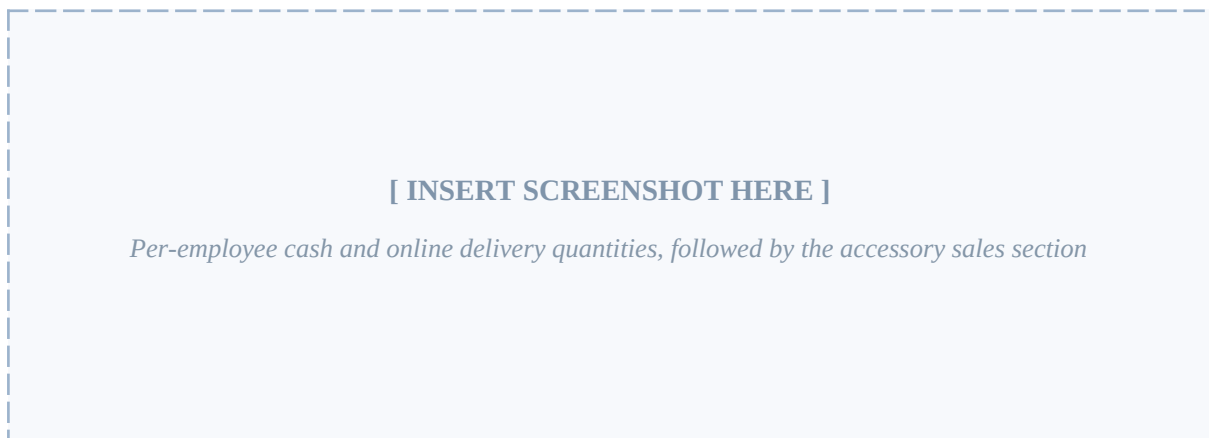


Fig 11: Daily Entry — Delivery Boy Split and Accessory Sales

4.4.4 Daily Entry — Expenses, Receipts and Credit

Three repeating-row sections capture the day's financial detail. Sundry expenses are recorded as description and amount pairs. Cheque and online receipts are recorded similarly. Credit sales are the most structured of the three, capturing the customer name, the product, the filled cylinders issued, the empty cylinders returned, the amount and free-text remarks, with the amount computed automatically from the product rate and quantity where appropriate. Vehicle expenses attribute a categorised cost to a specific vehicle in the fleet, and salary payments attribute an amount to a specific employee, a payment type and a salary month.

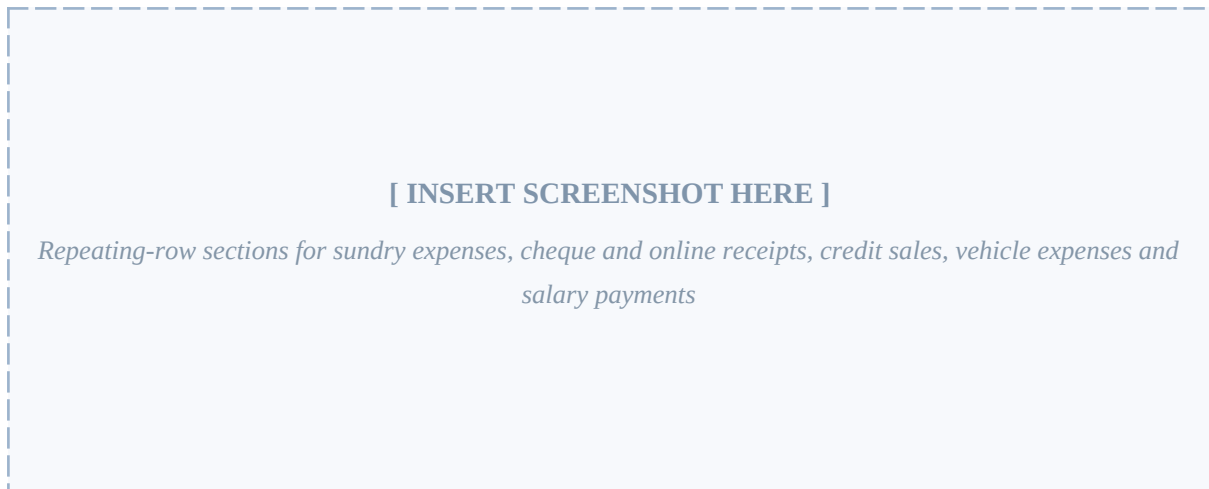


Fig 12: Daily Entry — Expenses, Cheque / Online and Credit Sales

4.4.5 Daily Entry — In-Out Stock Master and Cash Summary

The final sections of the daily entry present the computed position. The In-Out Stock Master displays, per product, the derived full-cylinder stock, the derived empty-cylinder stock and the shortage reminder. Alongside it, the godown stock section captures the physically counted figures so that the operator can compare computed and actual positions at a glance. The cash summary presents the complete derivation of closing cash on hand from opening cash through every receipt and disbursement category.

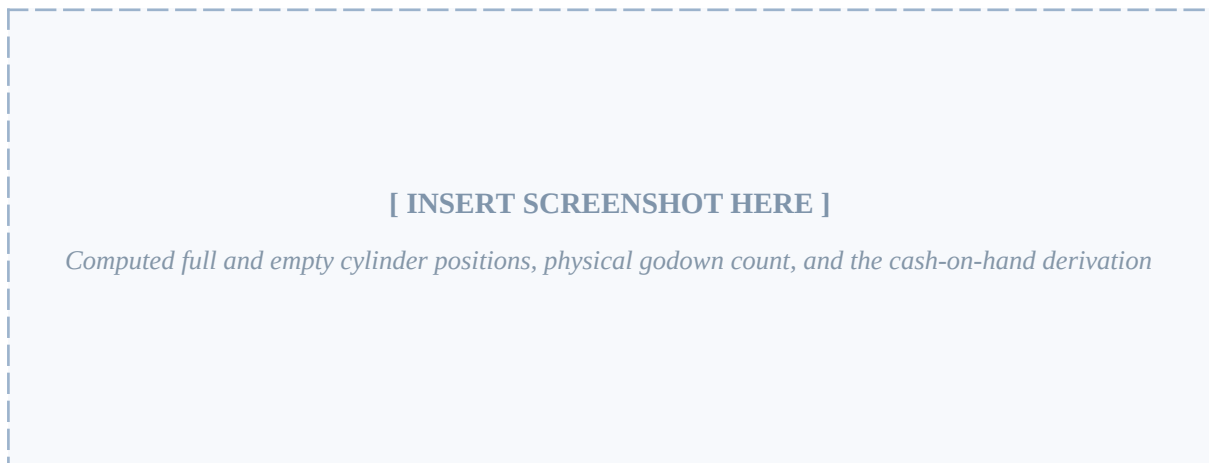


Fig 13: Daily Entry — In-Out Stock Master and Cash Summary

4.4.6 History and Day Reporting

The history tab presents every recorded day in reverse chronological order as a consolidated register, showing sales, expenses, credit, cash on hand and the In-Out stock position for each date. Selecting a row opens that day in the entry screen, read-only for the counter operator and editable for an administrator. Administrators additionally have a delete action on each row,

which removes the operational records for that date while preserving the permanent credit and payroll records.

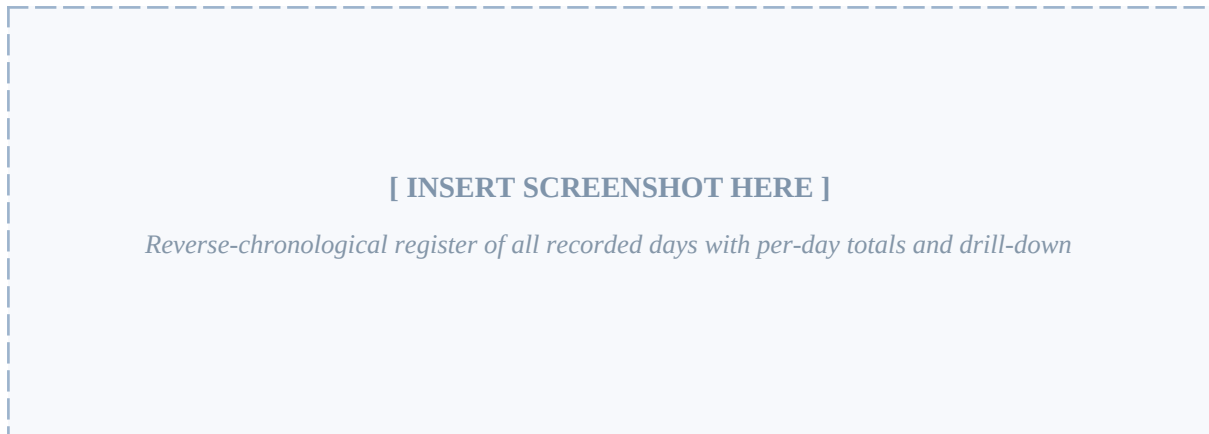


Fig 14: History Tab — Consolidated Day Register

4.4.7 Pending Credits and Recovery Recording

The pending credits screen lists every uncleared ledger row with its customer, original amount, cumulative recovered amount and derived outstanding balance, together with a badge indicating any empty cylinders still held against that credit. Recording a recovery captures the amount, the date, an optional note and any empty cylinders returned with the payment; when cumulative recoveries meet or exceed the original amount the row clears itself automatically and leaves the pending list.

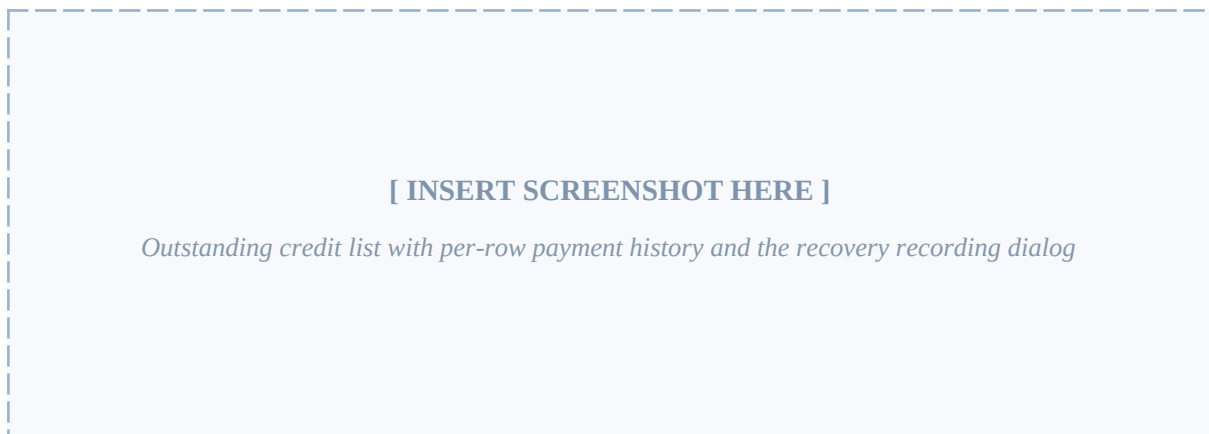


Fig 15: Pending Credits and Recovery Recording

4.4.8 Summary and Analysis

The summary screen aggregates the recorded data over a selectable period of one week, the current month, or all time. It reports total sales, total expenses, outstanding credit, cheque and online receipts, delivery volume and salary disbursement, together with a per-product breakdown showing quantities split by channel and the revenue attributable to each.

[INSERT SCREENSHOT HERE]

Period selector with aggregate totals and the per-product channel breakdown

Fig 16: Summary Tab — Period Sales Analysis

4.4.9 Administrative Dashboard

The administrative dashboard presents the management view: month-to-date sales with a percentage comparison against the equivalent period of the preceding month, cylinders sold, total outstanding credit across the portfolio, and commission earned computed at the applicable commission rate. Below the indicators sit a product snapshot showing quantity, rate, revenue and commission per product, and a recent-days panel showing sales, cylinder count and closing cash for the latest trading dates.

[INSERT SCREENSHOT HERE]

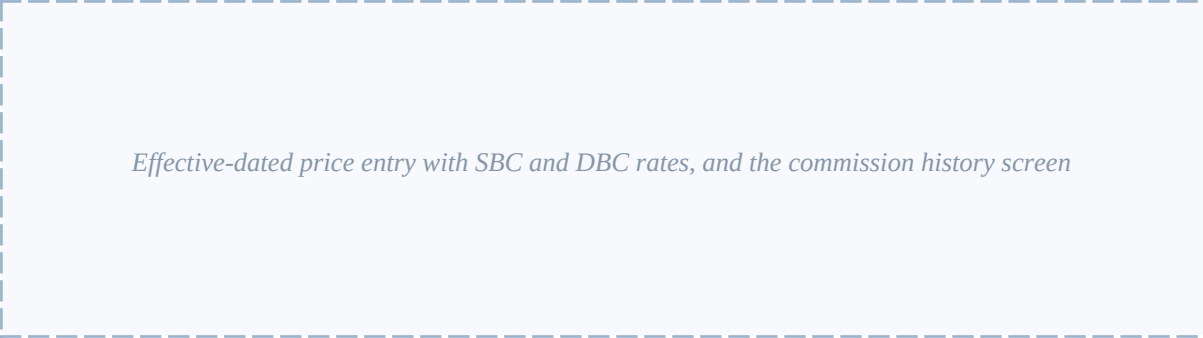
Month-to-date KPI cards, product snapshot table and recent trading days panel

Fig 17: Admin Dashboard — Key Performance Indicators

4.4.10 Rate Administration

The price history screen allows an administrator to record a new effective-dated rate for any product, capturing the standard rate together with the independent SBC and DBC rates and a note recording the reason for the revision. The commission screen operates identically for the distributor commission per cylinder. Because both are histories rather than mutable values, adding a new rate never disturbs the computed revenue of any previously recorded day.

[INSERT SCREENSHOT HERE]

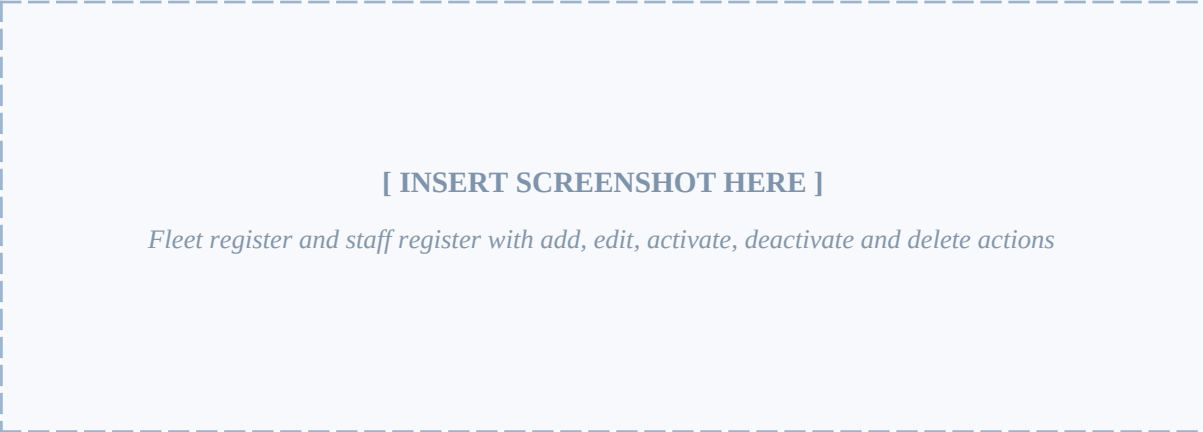


Effective-dated price entry with SBC and DBC rates, and the commission history screen

Fig 18: Admin — Price History and Commission Management

4.4.11 Master Data Management

The vehicle master maintains the fleet register with registration number, type, capacity and notes, supporting activation and deactivation so that a retired vehicle disappears from selection lists without orphaning the historical expense rows that reference it. The employee master operates on the same principle, additionally holding the monthly base salary that the payroll module uses as the entitlement against which advances and salary payments are measured, and the role field that determines which employees appear in the delivery section.



[INSERT SCREENSHOT HERE]

Fleet register and staff register with add, edit, activate, deactivate and delete actions

Fig 19: Admin — Vehicle Master and Employee Master

4.4.12 Administrative Reporting

Three administrative reporting screens complete the interface. The day report renders a complete printable statement for any single trading date, including product movement, revenue, commission entitlement, expenses, credit activity and the cash reconciliation. The salary report shows, per employee for the applicable salary month, the base salary, advances drawn, salary paid, total paid and the resulting balance, with over-advanced employees highlighted. The credit ledger presents every credit sale ever recorded, cleared and uncleared alike, with its complete recovery history.

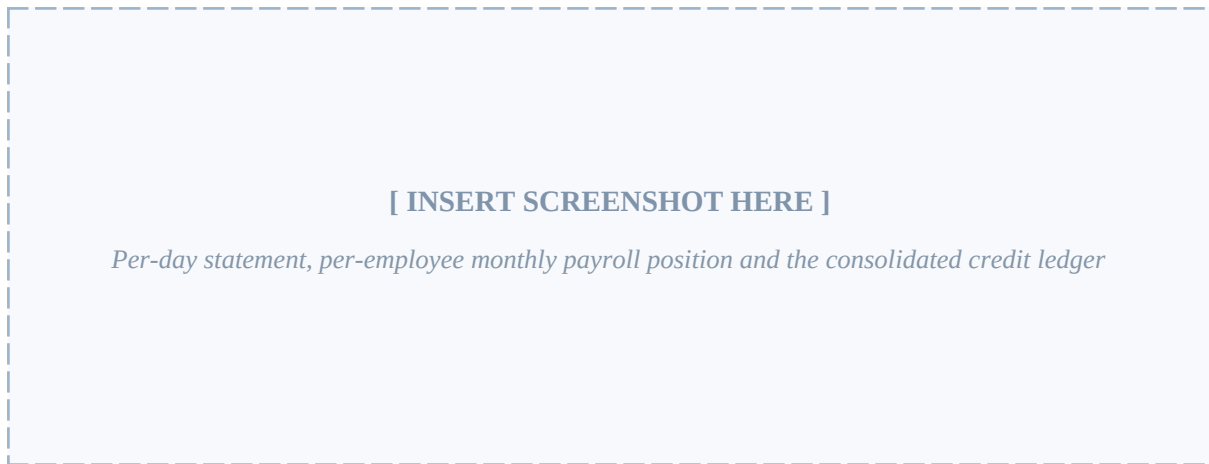


Fig 20: Admin — Day Report, Salary Report and Credit Ledger

4.5 Testing

Testing proceeded on three levels. Unit-level verification targeted the calculation engine, where the business formulae reside and where an error would corrupt data silently. Integration-level verification targeted the transactional persistence path and the authentication pipeline. System-level verification consisted of parallel running, in which the application and the existing manual register were maintained side by side for a period and their outputs compared each evening.

4.5.1 Test Strategy

Black-box functional testing was applied to each screen against the functional requirements enumerated in Section 2.7.1. White-box testing was applied to the calculation engine, where the internal branch structure — particularly the arrival, credit and same-day-netting conditions — was exercised directly with constructed inputs. Boundary testing addressed empty entries, single-product days, days with no vehicle arrival, days on which a credit was extended and settled together, and month-boundary payroll payments. Negative testing addressed invalid credentials, expired and tampered tokens, malformed date parameters and deliberate mid-transaction failures.

Parallel running was the decisive test. For the verification period the counter recorded each day in both the register and the application, and each evening the closing cylinder stock, the closing cash on hand and the outstanding credit total were compared. Any divergence was investigated to root cause before the period was allowed to continue.

4.5.2 Test Cases

Table 15: Unit and Integration Test Cases

ID	Test Case	Expected Result	Status
T-01	Log in with valid administrator credentials	Token issued with one-hour expiry; administrative portal with eleven tabs mounted	Pass
T-02	Log in with an invalid password	Authentication rejected; inline error displayed; no token stored	Pass
T-03	Submit a request to a protected endpoint with no token	Request rejected with an authorisation error	Pass
T-04	Submit a request with a tampered token signature	Signature verification fails; request rejected	Pass
T-05	Submit a request with an expired token	Distinct expiry response returned; client clears storage and prompts re-authentication	Pass
T-06	Restart the application with a valid stored token	Session verified against the server; portal mounted without re-login	Pass
T-07	Consolidated master load after authentication	Prices, commissions, employees, vehicles, ledger and all entries returned in one response	Pass
T-08	Record a day with cash sales only	Closing stock reduced by the sold quantity; cash on hand increased by the sale value	Pass
T-09	Record a day with a vehicle arrival	Closing stock increased by filled received; empty balance reduced by empties despatched	Pass
T-10	Record SBC and DBC quantities	Each channel valued at its own independent rate; both reduce closing stock	Pass
T-11	Verify opening stock carry-forward	Opening stock of a new day equals the computed closing stock of the previous day for every product	Pass
T-12	Record a credit sale with filled and empty quantities	Filled cylinders reduce closing stock; empty cylinders increase the empty balance; ledger row created	Pass
T-13	Record a partial recovery against a credit	Outstanding balance reduced; ledger row remains uncleared and stays in the pending list	Pass
T-14	Record a recovery equal to the outstanding balance	Ledger row marked cleared automatically and removed from the pending list	Pass
T-15	Record a credit and settle it the same day	Day reports zero outstanding credit; cash sales figure left unadjusted to match the physical drawer	Pass
T-16	Attempt to remove a credit sale that carries a payment	Ledger row retained; payment history preserved	Pass
T-17	Remove a credit sale that carries no payment	Ledger row deleted cleanly	Pass
T-18	Record empties returned alongside a recovery	Empty balance increased by the returned quantity on the payment date	Pass
T-19	Verify the cash-on-hand derivation	Computed figure equals opening cash plus all receipts less all disbursements including bank deposit	Pass
T-20	Verify online sales treatment in the cash formula	Online sales included in revenue but excluded from physical cash	Pass
T-21	Record a salary advance in the first	Salary month defaults to the previous month; report	Pass

ID	Test Case	Expected Result	Status
	ten days of a month	attributes the payment to that month	
T-22	Override the defaulted salary month	Stored month reflects the override rather than the payment date	Pass
T-23	Verify per-employee payroll balance	Balance equals base salary less advances and salary paid for the applicable month	Pass
T-24	Add a new effective-dated price	New rate applies to subsequent days; previously recorded days retain their original valuation	Pass
T-25	Verify commission computation for a past month	Commission calculated at the rate in force during that month, not the current rate	Pass
T-26	Deactivate a delivery employee	Employee removed from the delivery section; historical delivery rows retained intact	Pass
T-27	Deactivate a vehicle	Vehicle removed from the expense selector; historical expense rows retained intact	Pass
T-28	Force a failure mid-transaction during a daily entry save	Transaction rolled back; database unchanged; no partial day persisted	Pass
T-29	Save the same daily entry twice	Second save replaces rather than duplicates; no duplicate rows created	Pass
T-30	Delete a daily entry as administrator	Operational records removed; credit ledger and payroll records preserved	Pass
T-31	Attempt to open a past date as a counter operator	Screen presented read-only; edits not permitted	Pass
T-32	Edit a back-dated entry as administrator	Edit permitted; after save the view remains on the edited date rather than returning to today	Pass
T-33	Submit a malformed date to the delete endpoint	Request rejected with a format error before any query executes	Pass
T-34	Restart the server against an older schema	Migration routine detects and applies missing columns and tables; start-up proceeds	Pass
T-35	Restart the server against a current schema	Migrations detected as already applied; no duplicate change attempted	Pass
T-36	Verify the empty-cylinder running balance	Computed balance equals cumulative empties received less cumulative empties despatched across all history	Pass
T-37	Compare computed stock against a physical godown count	Computed and counted figures agree; any difference surfaced rather than absorbed	Pass
T-38	Render the application on a mobile viewport	Layout adapts; all daily entry sections remain usable	Pass

4.6 Results

The system was deployed to managed cloud infrastructure and placed into daily operational use at the distributorship. The results below cover measured performance, functional correctness established during parallel running, and the operational impact reported by the business.

4.6.1 Performance

Table 16: Performance Measurements

Operation	Observed	Target	Assessment
Initial bundle load over mobile broadband	under 2 seconds	under 3 seconds	Met
Startup session verification	under 300 milliseconds	under 1 second	Met
Consolidated master load (full dataset)	under 2 seconds	under 3 seconds	Met
Daily entry save (twelve-step transaction)	under 1 second	under 2 seconds	Met
Credit recovery recording	under 500 milliseconds	under 2 seconds	Met
Tab navigation after master load	effectively instantaneous	under 500 milliseconds	Met
Derived calculation refresh on keystroke	no perceptible delay	imperceptible	Met
Master data operation (vehicle or employee)	under 500 milliseconds	under 2 seconds	Met

The performance profile is dominated by the single consolidated load, which is precisely the intended outcome of that design decision. Because the entire working dataset is resident in client memory after that one request, every subsequent interaction is a local computation and the interface remains responsive irrespective of connection quality — a property that matters considerably at a counter served by mobile broadband.

4.6.2 Functional Correctness

During parallel running the system's derived figures were compared each evening against the manually maintained register. The closing full-cylinder stock agreed for every product on every day of the verification period. The closing cash on hand agreed with the physical cash count, with the only discrepancies traced to entries the operator had not yet recorded rather than to computational error. The outstanding credit total agreed with a manual summation of the khata.

Two findings from parallel running are worth recording. First, on two occasions the system and the register disagreed and investigation established that the register was wrong — an arithmetic slip in a carried-forward figure that had gone unnoticed. This is the failure mode the project set out to eliminate, and its detection during testing was the clearest available validation of the approach. Second, the empty-cylinder balance had no counterpart in the register to

compare against, because the manual system never produced that figure; it was instead validated directly against a physical godown count, which it matched.

4.6.3 Operational Impact

Table 17: Operational Impact Assessment

Dimension	Before	After
End-of-day reconciliation time	40 to 70 minutes of manual totalling and cross-checking	Data entry only; derived figures available immediately on save
Stock arithmetic errors	Periodic, undetected until a full physical count	Eliminated; the chain is computed and carried forward automatically
Empty-cylinder position	Not available at all	Available continuously per product, validated against physical count
Outstanding credit visibility	Required manual summation; effectively never computed portfolio-wide	Displayed on the dashboard at all times and per customer on demand
Empties held by credit customers	Untracked	Recorded per ledger row and reduced as empties are returned
Historical rate fidelity	Lost; only amounts retained	Full effective-dated history; past days re-priceable at their own rates
Payroll position	Informal notes of advances	Per-employee balance against base salary for the applicable salary month
Per-vehicle running cost	Not separable from general expenses	Categorised and attributed to individual vehicles
Remote management access	None	Any browser, any device, over HTTPS
Data durability	Single paper copy at one location	Managed database with provider-level backup and redundancy
Access control	Physical possession of the register	Token-based authentication with distinct operator and administrator capabilities

4.6.4 Summary of Results

Measured against the eight objectives set out in Section 1.3, the outcome is as follows. The single atomic unit of record is established and verified by deliberate mid-transaction failure testing. Manual stock arithmetic is eliminated and the carry-forward verified across the full history. The empty-cylinder cycle is reconciled and validated against a physical count. Outstanding credit is continuously visible at both portfolio and customer level. Historical pricing fidelity is preserved through effective-dated history and verified by confirming that a rate revision leaves prior days undisturbed. Role-based boundaries are enforced and verified. Management reporting is delivered across five distinct reporting surfaces. The system is deployed and in sustained daily use. All eight objectives are met.

Conclusion and Future Work

Conclusion

This project set out to replace the handwritten daily accounting of a working LPG distributorship with a single transactional system of record, and it has done so. The resulting application is not a demonstration or a prototype: it runs on managed cloud infrastructure, it is used every trading day by the staff and proprietor of Jay Ranchhod Gas Service, and it is the authoritative record of that agency's stock, cash and credit position.

The technical achievement rests on a small number of decisions that proved sound under operational load. Modelling one trading day as one atomic transaction spanning twelve tables removed an entire category of failure, in which a partially recorded day leaves the books in a state from which recovery requires guesswork. Deriving the closing stock in software and carrying it forward automatically eliminated the propagating arithmetic error that was the single most damaging characteristic of the manual system — a claim substantiated during parallel running, when the system twice disagreed with the register and the register proved to be wrong.

Computing the empty-cylinder balance as a running total across the complete history of trading gave the business a figure it had never previously possessed. The empty-cylinder cycle is the defining characteristic of LPG distribution and the aspect of it that generic inventory software does not model, and reconciling it required drawing together three sources — ordinary sale exchanges, empties recorded against credit sales, and empties returned with later recoveries — that the manual system recorded in three unconnected places and netted nowhere.

Storing rates and commissions as effective-dated histories rather than as mutable current values preserved the ability to evaluate any past day at the rate that actually applied to it, which in a sector where prices are revised by external authority on an irregular schedule is not a refinement but a requirement. Partitioning access into two roles gave the counter what it needs to record the day without giving it the ability to alter closed days or master configuration.

Beyond the technical outcome, the project produced a clear lesson about software for small businesses. The greatest risk was never technical; it was that a working counter would reject a system that asked it to change its habits. That risk was managed by a deliberate design commitment: the daily entry screen follows the section order of the physical register it replaces, it asks for nothing the operator does not already know at the moment of entry, and it shows its arithmetic rather than hiding it. Adoption followed from those choices at least as much as from any architectural decision.

The application also demonstrates that the contemporary web platform is now entirely adequate for delivering business-critical software to a small enterprise. A React single-page application, a Node service and a managed relational database — none of them exotic, all of them freely licensed — deliver a system that loads in under two seconds over mobile broadband, runs identically on a counter desktop and a proprietor's telephone, requires no installation, and updates for every user the moment a new build is deployed. For a business of this size, that combination was not available at an accessible price point a decade ago.

Future Work

Several extensions have been identified, some arising from the development process and others requested by the business after live use began.

Analytical and Forecasting Capability

The system now holds a growing corpus of structured daily trading data. That data supports demand forecasting: seasonal consumption patterns for domestic cylinders, weekday and weekend variation in commercial offtake, and the lead-time relationship between plant arrivals and stock-out risk. A forecasting module could recommend indent quantities and warn when projected stock will fall below a safety threshold before the next scheduled arrival.

Customer Master and Credit Scoring

Credit customers are currently identified by a normalised name string. Promoting customers to a first-class master entity with contact details, credit limit and payment history would enable ageing analysis, automated payment reminders, and a simple internal credit score derived from settlement behaviour, allowing credit limits to be set on evidence rather than on impression.

Offline-First Operation

Although the consolidated load makes the application resilient to slow connections, it cannot presently record an entry with no connection at all. Implementing a service worker with a local write queue would allow the counter to continue recording through a connectivity outage, synchronising when the connection returns. For a retail counter this is a material robustness improvement.

Native Mobile Application

The responsive interface works on mobile browsers, but a packaged application would enable push notifications for credit-recovery reminders, camera capture for expense receipts, and device-level biometric unlock, all of which have been requested by the proprietor.

Delivery Route Optimisation

The system records which delivery employee delivered how many cylinders but not where. Capturing delivery addresses and coordinates would enable route planning, per-route cost analysis and delivery-time estimation, extending the system from record-keeping into operational optimisation.

Accounting Integration and Statutory Output

The system is deliberately an operational record rather than a book of accounts. Adding an export in the interchange formats accepted by common Indian accounting packages would eliminate the duplicate data entry currently performed by the agency's accountant, and would allow GST-relevant summaries to be generated directly from primary transaction data.

Serial-Level Cylinder Tracking

Aggregate empty-cylinder balance tracking identifies that cylinders are missing but not where they went. Should the economics of barcode or RFID marking become favourable, serial-level tracking would permit individual custody attribution and materially reduce cylinder loss, at the cost of a scanning discipline at every touchpoint.

Multi-Branch Operation

The data model presently assumes a single distributorship with a single godown. Extending it to support multiple branches with inter-branch stock transfers and consolidated reporting would allow the system to serve a distributor operating several agencies, which is a common structure in the sector.

Automated Backup and Audit Trail

While the managed database provider performs backups, an application-level scheduled export to independent storage would protect against provider-level failure and provide an accessible archive. Separately, an append-only audit log recording who changed what and when — particularly for administrative edits to back-dated entries — would strengthen accountability in a system that permits historical correction by design.

References

1. R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Pearson Education, 2016. — Relational design, normalisation and transaction theory underpinning the nineteen-table schema.
2. A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed. McGraw-Hill Education, 2019. — ACID properties, isolation levels and concurrency control applied to the daily-entry transaction.
3. MySQL 8.0 Reference Manual, Oracle Corporation. Available: <https://dev.mysql.com/doc/refman/8.0/en/> — InnoDB transactional storage, foreign key semantics and the INSERT ... ON DUPLICATE KEY UPDATE construct.
4. InnoDB Storage Engine Documentation, Oracle Corporation. Available: <https://dev.mysql.com/doc/refman/8.0/en/innodb-storage-engine.html> — Row-level locking and crash recovery guarantees relied upon by the persistence layer.
5. React Documentation, Meta Platforms Inc. Available: <https://react.dev/> — Component model, hooks, state management and rendering behaviour of the presentation tier.
6. Vite Documentation. Available: <https://vite.dev/> — Development server, hot module replacement and production bundling configuration.
7. Node.js Documentation, OpenJS Foundation. Available: <https://nodejs.org/en/docs/> — Event loop, asynchronous input-output model and runtime behaviour of the application tier.
8. Express 5.x API Reference, OpenJS Foundation. Available: <https://expressjs.com/en/5x/api.html> — Routing, middleware composition and request handling.
9. mysql2 Documentation. Available: <https://sidorares.github.io/node-mysql2/docs> — Promise interface, connection pooling and prepared-statement support.
10. M. Jones, J. Bradley and N. Sakimura, JSON Web Token (JWT), RFC 7519, Internet Engineering Task Force, May 2015. Available: <https://datatracker.ietf.org/doc/html/rfc7519> — Token structure, claim set and signature verification.
11. M. Jones, JSON Web Algorithms (JWA), RFC 7518, Internet Engineering Task Force, May 2015. Available: <https://datatracker.ietf.org/doc/html/rfc7518> — The HS256 signing algorithm used for session tokens.
12. OWASP Top Ten Web Application Security Risks, Open Worldwide Application Security Project. Available: <https://owasp.org/www-project-top-ten/> — Injection prevention, authentication and access-control guidance applied to the API surface.
13. OWASP Authentication Cheat Sheet, Open Worldwide Application Security Project. Available:

- https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html — Session lifetime and credential-handling recommendations.
14. Cross-Origin Resource Sharing (CORS), MDN Web Docs, Mozilla. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS> — Cross-origin policy governing the separately hosted frontend.
 15. Using the Fetch API, MDN Web Docs, Mozilla. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch — Request construction and response handling in the service layer.
 16. Window.localStorage, MDN Web Docs, Mozilla. Available: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage> — Client-side credential persistence across sessions.
 17. M. Fowler, Patterns of Enterprise Application Architecture. Addison-Wesley, 2002. — Layered architecture, unit of work and the transaction script pattern applied to the entry handler.
 18. E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley, 2003. — Aggregate boundaries informing the treatment of a daily entry as a single consistency unit.
 19. R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017. — Dependency direction and the isolation of business rules in the calculation module.
 20. R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. McGraw-Hill Education, 2019. — Requirement engineering, feasibility assessment and the testing strategy adopted in Section 4.5.
 21. I. Sommerville, Software Engineering, 10th ed. Pearson Education, 2015. — Process model, requirement specification and system-design methodology.
 22. G. Booch, J. Rumbaugh and I. Jacobson, The Unified Modeling Language User Guide, 2nd ed. Addison-Wesley, 2005. — Use case, sequence and class modelling conventions used in Section 3.4.
 23. T. DeMarco, Structured Analysis and System Specification. Yourdon Press, 1979. — Data flow diagram notation and levelled decomposition used for the context and level 1 diagrams.
 24. Petroleum and Natural Gas Regulatory Board, Government of India. Available: <https://www.pngrb.gov.in/> — Regulatory framework governing LPG distribution in India.
 25. Bharat Petroleum Corporation Limited — LPG Distributor Information. Available: <https://www.bharatpetroleum.in/> — Distributor commission structure, cylinder variants and scheme definitions.

26. Ministry of Petroleum and Natural Gas, Government of India. Available: <https://mopng.gov.in/> — Policy context for domestic LPG pricing and subsidy schemes.
27. S. Chopra and P. Meindl, Supply Chain Management: Strategy, Planning and Operation, 6th ed. Pearson Education, 2015. — Inventory theory and the returnable container problem underlying the empty-cylinder model.
28. R. H. Ballou, Business Logistics / Supply Chain Management, 5th ed. Pearson Education, 2004. — Reverse logistics and returnable transport item management.
29. M. Fleischmann et al., "Quantitative Models for Reverse Logistics: A Review," European Journal of Operational Research, vol. 103, no. 1, pp. 1–17, 1997. — Modelling approaches for returnable asset cycles.
30. SweetAlert2 Documentation. Available: <https://sweetalert2.github.io/> — Modal confirmation and notification dialogs used for save, delete and session-expiry events.
31. ESLint Documentation, OpenJS Foundation. Available: <https://eslint.org/docs/latest/> — Static analysis configuration including the React hooks rule set.
32. Render Documentation. Available: <https://render.com/docs> — Managed application hosting, environment configuration and deployment pipeline.
33. Aiven for MySQL Documentation. Available: <https://aiven.io/docs/products/mysql> — Managed database provisioning, SSL enforcement and backup policy.
34. HTTP Semantics, RFC 9110, Internet Engineering Task Force, June 2022. Available: <https://datatracker.ietf.org/doc/html/rfc9110> — Method semantics and status code usage across the REST interface.
35. Git Documentation, Software Freedom Conservancy. Available: <https://git-scm.com/doc> — Version control workflow used throughout development.
36. W3C Web Content Accessibility Guidelines (WCAG) 2.1. Available: <https://www.w3.org/TR/WCAG21/> — Contrast, labelling and keyboard-navigation guidance applied to the interface.